

30-Day High-Yield Battle Plan

Front Office Technical Associate (FOTA)
D.E. Shaw India — Campus Placement Preparation

Candidate Profile: Systems-focused | Intermediate Python/C++/Linux

Preparation Window: 20-30 Days

Methodology: 80/20 Rule | Sprint-Based

Target Date: July 2026

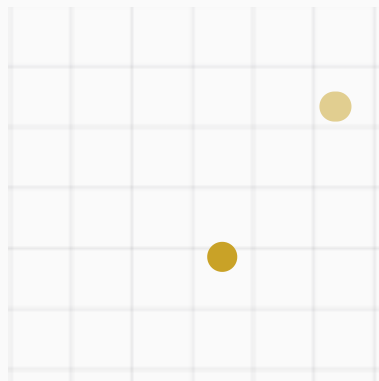


Table of Contents

Prelude: Project Architecture & Strategy

Week 1: Project Defensibility & Low-Level Diagnostics

Day-by-Day Micro-Schedule

Core Concepts to Master

Practical Tasks & Drills

Actionable Learning Resources

Interview Scenarios & Mock Diagnostics

Edge-Case Interview Questions

Week 2: Automation Scripting, Data Parsing & Core Logic

Day-by-Day Micro-Schedule

Core Concepts to Master

Practical Tasks & Drills

Actionable Learning Resources

Interview Scenarios & Mock Diagnostics

Edge-Case Interview Questions

Week 3: The Coding Baseline & Data Retrieval Pipelines

Day-by-Day Micro-Schedule

Core Concepts to Master

Practical Tasks & Drills

Actionable Learning Resources

Interview Scenarios & Mock Diagnostics

Edge-Case Interview Questions

Week 4: Full Production Simulation & Interview Polish

Day-by-Day Micro-Schedule

Core Concepts to Master

Practical Tasks & Drills

Actionable Learning Resources

Interview Scenarios & Mock Diagnostics

Edge-Case Interview Questions

Capstone Project: ProductionOps Dashboard

Appendix A: Behavioral Question Bank

Appendix B: Linux Command Quick Reference

Appendix C: Complexity Cheat Sheet

Prelude: Project Architecture & Preparation Strategy

Target Role: Front Office Technical Associate (FOTA) — D.E. Shaw India | **Preparation Window:** 20-30 Days | **Methodology:** 80/20 Rule, Sprint-Based | **Candidate Profile:** Systems-focused, Intermediate Python/C++/Linux, Beginner DSA

Your Project Architecture at a Glance (Interviewer's Perspective)

Project	Tech Stack	Production-Relevant Mechanism	What D.E. Shaw Will Probe
UnMTP	Python, Sockets, Tar, sendfile()	Zero-copy kernel streaming via socket.sendfile(), chunked tar piping, 8MB socket buffers, cross-platform path normalization	Kernel bypass, buffer sizing, network throughput optimization, syscall mechanics
Runux	Python (CLI), Shell	NL-to-shell translation, command sandboxing, blast-radius analysis, memory-backed execution state, plugin hooks	Process isolation, shell injection safety, signal handling, command pipeline design
The-Stickerist	Node.js, Puppeteer, Sharp	Iterative quality adjustment, zombie process cleanup, session persistence, headless browser resource management	OOM prevention, iterative streaming, resource cleanup patterns, exception handling

How This Plan Is Structured

Each week is a self-contained **Sprint** with: (1) Daily micro-schedule with exact hours and theory-vs-coding splits, (2) Deep technical concepts with point-by-point mechanisms, (3) Practical exercises with full implementations, (4) Exact learning resources (specific YouTube videos, documentation pages), (5) D.E. Shaw interview scenarios with structured response strategies, and (6) Edge-case interview questions on every topic.

Recommended Daily Time Commitment: 6-8 hours/day. If you have only 20 days, compress by doing Days 1-2 together and skipping the capstone build (Days 29-30). If you have 30 days, build the capstone — it is your strongest interview asset.

Week 1: Project Defensibility & Low-Level Diagnostics

Sprint Goal: Make every line of your three projects defensible. Master Linux internals, C++ OOPs, Git mechanics, and system diagnostics at a level where you can whiteboard the architecture under pressure.

Day-by-Day Micro-Schedule

Day 1 — UnMTP Architecture & Socket Programming Deep Dive (6 hours)

Time Block	Activity
2 hours	Theory: <code>socket.sendfile()</code> syscall mechanics, kernel zero-copy, TCP socket buffers, struct module for network packing
2 hours	Theory: Tar streaming (<code>mode='w '</code>), file descriptor lifecycle, Python's socket module internals
2 hours	Coding: Rewrite the UnMTP host's core transfer loop from memory; benchmark <code>sendfile()</code> vs. traditional <code>read()/write()</code>

Day 2 — Runux Execution Engine & Signal Handling (6 hours)

Time Block	Activity
2 hours	Theory: Python signal handling (SIGINT, SIGTERM), <code>BaseException</code> vs. Exception hierarchy, process isolation
2 hours	Theory: Shell command injection vectors, sandboxing approaches, <code>subprocess</code> module security
2 hours	Coding: Implement a mini command runner with <code>subprocess.run()</code> , timeout handling, and safe argument passing

Day 3 — Linux File Descriptors, Standard Streams & Process Management (7 hours)

Time Block	Activity
2 hours	Theory: File descriptors (0/1/2), <code>dup()/dup2()</code> , pipes, named pipes (FIFOs), <code>procfs</code>
2 hours	Theory: Process states (R/S/D/T/Z), <code>fork()/exec()</code> , zombie vs. orphan processes, <code>wait()/waitpid()</code>
2 hours	Theory: Environment variables (PATH, HOME, LD_PRELOAD), shell startup files, export semantics
1 hour	Coding: Write a bash script that monitors process states using <code>/proc/<pid>/stat</code>

Day 4 — C++ OOPs & Memory Management for Interviews (6 hours)

Time Block	Activity
2 hours	Theory: Classes/objects, inheritance (single/multiple), polymorphism (vtable, dynamic dispatch), encapsulation, abstraction
2 hours	Theory: Constructors/destructors, copy constructor, assignment operator, Rule of Three/Five, RAII
2 hours	Coding: Implement a RAII wrapper for a file descriptor; implement a simple smart pointer (like <code>unique_ptr</code>)

Day 5 — Git Internals & Version Control Mechanics (5 hours)

Time Block	Activity
2 hours	Theory: Git object model (blob, tree, commit, tag), <code>.git/</code> directory structure, SHA-1 addressing, packfiles
2 hours	Theory: Branching (ref files), merging (fast-forward, 3-way, octopus), rebasing, cherry-picking, reflog
1 hour	Coding: Walk through <code>.git/objects/</code> after a commit; resolve a simulated merge conflict manually

Day 6 — System Diagnostics Basics + Log Analysis (6 hours)

Time Block	Activity
2 hours	Theory: Linux performance tools (<code>top</code> , <code>htop</code> , <code>vmstat</code> , <code>iostat</code> , <code>netstat</code> , <code>ss</code> , <code>df</code> , <code>du</code> , <code>free</code>)
2 hours	Theory: Log file anatomy (<code>syslog</code> , <code>journald</code>), log rotation, <code>journalctl</code> , <code>dmesg</code> , <code>/var/log/</code> structure
2 hours	Coding: Write a Python script that parses <code>/var/log/syslog</code> , extracts ERROR lines by timestamp, and reports frequency by hour

Day 7 — Week 1 Consolidation & Mock Review (5 hours)

Time Block	Activity
3 hours	Review all Week 1 notes; re-implement UnMTP's transfer handshake without looking at source
2 hours	Write out answers to 5 likely interview questions about your projects (see scenarios below)

Core Concepts to Master

1. UnMTP Socket & Transfer Mechanics

socket.sendfile() — Zero-Copy Kernel Streaming:

- The sendfile() syscall (Linux 2.1+) copies data directly from a file descriptor to a socket fd **without passing through userspace**.
- Traditional path: disk → kernel page cache → read() buffer (userspace) → write() buffer (userspace) → kernel socket buffer → NIC.
- Zero-copy path: disk → kernel page cache → kernel socket buffer → NIC (userspace never touches the data).
- Python's socket.sendfile() wraps this syscall; available from Python 3.3+.
- Requirements: **output socket must be TCP** (SOCK_STREAM); file must be a regular file (not a pipe).
- Limitations: Cannot transform data mid-flight (no encryption, no compression); platform-dependent behavior on non-Linux.

The 5GB Threshold Logic:

```
if file_size >= BIG_FILE_THRESHOLD:
    # Zero-copy path: direct kernel streaming
    socket.sendfile(file)
else:
    # Userspace path: streaming tar archive
    tar = tarfile.open(fileobj=socket, mode='w|')
    tar.add(file)
```

- Why 5GB? This is a heuristic. Small files benefit from tar bundling (reduces round-trips, preserves metadata). Large files would cause excessive memory pressure if tar'd.
- The `w|` mode means **streaming tar** — it writes the archive structure incrementally without seeking back. This is critical for socket-based transfer because sockets are not seekable.

8MB Socket Buffer Tuning:

```
sock.setsockopt(socket.SOL_SOCKET, socket.SO_SNDBUF, 8 * 1024 * 1024)
```

- Default socket buffers are typically 212KB (Linux auto-tuning). For gigabit Wi-Fi, larger buffers smooth out burstiness.
- Trade-off: Larger buffers increase latency (bufferbloat) but improve throughput on high-BDP (Bandwidth-Delay Product) links.

struct.pack() for Network Protocols:

- UnMTP uses `struct.pack('!Q', file_size)` to send the 8-byte file size header.
- `!` = network byte order (big-endian); `Q` = unsigned long long (8 bytes).
- The receiver uses `struct.unpack('!Q', raw_bytes)` to decode — a classic binary protocol pattern.

2. Runux Execution Architecture

Signal Handling Contract:

- `KeyboardInterrupt` inherits from `BaseException`, NOT `Exception`.
- `except Exception:` will **not** catch `Ctrl+C` — this is a deliberate design choice.
- Inner functions re-raise `KeyboardInterrupt` after cleanup so it always bubbles to `main()`.
- Pattern: `cleanup(); raise` inside inner handlers ensures no orphaned processes.

The subprocess Module Security Model:

- `subprocess.run(shell=True)` is dangerous — it invokes `/bin/sh -c <cmd>` which enables shell injection.
- Safe pattern: pass command as a list: `subprocess.run(['ls', '-la', '/path'])` — no shell parsing occurs.
- The shell does wildcard expansion (*), variable substitution (\$VAR), and command substitution (`cmd`).
- Runux must sanitize natural-language-derived commands before execution — this is a critical security boundary.

Plugin Hook Architecture:

- Runux implements a plugin system with three hooks: `run_before_translate`, `run_command_generated`, `run_after_execute`.
- This is an **observer pattern** — allows extending functionality without modifying core code.
- Production relevance: Logging plugins, audit plugins, safety-check plugins can be injected.

3. Linux/Unix Fundamentals — The Complete Mechanism List

File Descriptors (FDs):

- Every open file/resource in Linux is referenced by an integer FD.
- FD 0 = `stdin`, FD 1 = `stdout`, FD 2 = `stderr`. When a process opens a file, the kernel assigns the lowest available FD (typically 3+).
- `dup2(oldfd, newfd)` copies the FD table entry — used for redirection: `dup2(fd, 1)` redirects `stdout` to a file.
- `pipe(fds)` creates a pair of FDs: `fds[0]` for reading, `fds[1]` for writing — the basis of all shell piping.

Process Lifecycle:

1. `fork()` creates a child process with a copy of the parent's address space (Copy-On-Write).
2. The child calls `execve()` to replace its memory image with a new program.
3. The parent calls `wait()` or `waitpid()` to reap the child's exit status.

4. If the parent dies first, the child is **orphaned** and reparented to init (PID 1).
5. If the child exits but the parent hasn't wait()'ed, the child becomes a **zombie** (state Z).

The procfs Filesystem (/proc):

Path	What It Shows
/proc/<pid>/cmdline	Command that started the process
/proc/<pid>/stat	Process state, PID, PPID, CPU usage, virtual memory size
/proc/<pid>/fd/	Directory of symlinks to all open FDs for this process
/proc/<pid>/status	Human-readable process info (Name, State, VmRSS, Threads)
/proc/<pid>/environ	Environment variables (null-separated)
/proc/loadavg	System load average (1min, 5min, 15min)
/proc/meminfo	Detailed memory statistics

Standard Streams & Redirection Mechanics:

- `command > file` — redirects FD 1 (stdout) to file using `dup2()`.
- `command 2>&1` — redirects FD 2 (stderr) to wherever FD 1 points.
- `command < file` — redirects FD 0 (stdin) from file.
- `command | command` — creates a pipe; left process writes to `fds[1]`, right process reads from `fds[0]`.
- Order matters: `command >file 2>&1` (both to file) is different from `command 2>&1 >file` (stderr to original stdout, stdout to file).

4. C++ OOPs — Interview-Ready Deep Dive

Pillar	Mechanism	C++ Implementation
Encapsulation	Data hiding, controlled access	private/protected/public access specifiers; getter/setter methods
Inheritance	Code reuse, IS-A relationship	class Derived : public Base; constructor chaining (Base() called before Derived())
Polymorphism	Same interface, different behavior	Virtual functions → vtable → dynamic dispatch; virtual keyword in base class
Abstraction	Hide complexity, expose essentials	Abstract classes (pure virtual functions = 0); interfaces

The Vtable Mechanism (Critical for Interviews):

- When a class has at least one virtual function, the compiler creates a **vtable** (virtual function table) — an array of function pointers.

- Each object of that class gets a hidden **vp_{tr}** (vtable pointer) as its first data member.
- `sizeof(Base)` increases by 8 bytes (on 64-bit) due to `vptr`.
- Dynamic dispatch: `obj->virtualFunc()` becomes `obj->vptr[0](obj)` at runtime.
- Pure virtual functions (= 0) make a class abstract — cannot instantiate.

RAII (Resource Acquisition Is Initialization):

- Acquire resource in constructor; release in destructor.
- Task: Write a `FileDescriptor` class that wraps an `int fd`; constructor calls `open()`, destructor calls `close()`.
- This prevents resource leaks even if exceptions are thrown.

Rule of Three/Five:

- If you define any of: destructor, copy constructor, copy assignment operator → you likely need all three (Rule of Three).
- C++11 adds: move constructor, move assignment operator (Rule of Five).
- Default copy is shallow — dangerous for pointer members (double-free).

5. Git Internals — Know the DAG**Git Object Model (4 object types):**

1. **Blob** — file content only (no filename, no permissions). Addressed by SHA-1 of content.
2. **Tree** — directory structure. Points to blobs (with filenames) and other trees. Like a directory listing.
3. **Commit** — points to a tree (root directory), parent commit(s), author, timestamp, message.
4. **Tag** — lightweight (just a ref) or annotated (a separate object with GPG signature).

Branch = Reference File:

- A branch is just a file in `.git/refs/heads/<branch-name>` containing a commit SHA-1.
- HEAD is a symbolic reference: `.git/HEAD` contains `ref: refs/heads/main`.
- Creating a branch: `echo <commit-sha> > .git/refs/heads/new-branch` (literally this simple).
- Detached HEAD: HEAD points directly to a commit SHA, not through a branch ref.

Merge Types:

- **Fast-forward**: Current branch tip is direct ancestor of target — just moves the pointer.
- **Three-way merge**: Two branches diverged; finds common ancestor (merge base), merges all three trees.
- **Octopus merge**: Merges more than two branches simultaneously.

6. System Diagnostics — Beginner to Interview-Ready

Tool	What it Shows	Key Flags
------	---------------	-----------

top / htop	Real-time processes, CPU%, memory	htop is interactive, color-coded
vmstat 1	CPU, memory, IO, processes, context switches	1 = 1-second interval
iostat -xz 1	Disk I/O per device, %utilization	-x extended, -z skip idle
ss -tulpn	Listening sockets, PIDs	-t TCP, -u UDP, -l listen, -p PID, -n numeric
df -h	Filesystem disk usage	-h human-readable
free -h	Memory usage	-h human-readable
ps aux --sort=-%cpu	All processes sorted by CPU	--sort for ordering
lsdf -p <pid>	All open files/FDs for a process	-p process filter
strace -p <pid>	System calls made by a process	-p attach, -e filter syscalls

Practical Tasks & Drills

Exercise 1: Zero-Copy Benchmark (UnMTP Defense)

Task: Write a Python script that benchmarks `socket.sendfile()` against manual `read()/write()` for transferring a 1GB file over localhost TCP.

```
#!/usr/bin/env python3
"""Benchmark: sendfile() vs read/write loop"""
import socket, tempfile, os, time

HOST, PORT = '127.0.0.1', 9999
FILE_SIZE = 1 * 1024 * 1024 * 1024 # 1 GB

def create_test_file(path, size):
    with open(path, 'wb') as f:
        chunk = os.urandom(8 * 1024 * 1024)
        written = 0
        while written < size:
            to_write = min(len(chunk), size - written)
            f.write(chunk[:to_write])
            written += to_write

def sender_sendfile(filepath):
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT)); server.listen(1)
    conn, addr = server.accept()
    filesize = os.path.getsize(filepath)
    conn.sendall(filesize.to_bytes(8, 'big'))
    with open(filepath, 'rb') as f:
        start = time.perf_counter()
        conn.sendfile(f) # ZERO-COPY PATH
        elapsed = time.perf_counter() - start
    conn.close(); server.close()
    return elapsed

def sender_readwrite(filepath):
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT)); server.listen(1)
    conn, addr = server.accept()
    filesize = os.path.getsize(filepath)
    conn.sendall(filesize.to_bytes(8, 'big'))
    with open(filepath, 'rb') as f:
        start = time.perf_counter()
        while True:
            chunk = f.read(8 * 1024 * 1024)
            if not chunk: break
            conn.sendall(chunk)
        elapsed = time.perf_counter() - start
    conn.close(); server.close()
    return elapsed
```

Expected Discussion Points: (1) Why does `sendfile()` show lower CPU usage but similar throughput on localhost? (2) What prevents using `sendfile()` with TLS/SSL encryption? (3) Why did we use `SO_REUSEADDR`? (4) What happens if the receiver is slower than the sender? (TCP backpressure)

Exercise 2: Process State Monitor (Runux/Linux Defense)

Task: Write a bash script that monitors all processes matching a given name and reports their states, CPU%, and memory usage by reading `/proc` directly.

```
#!/bin/bash
# proc_monitor.sh – Direct /proc inspection for process diagnostics

TARGET="$1"
[[ -z "$TARGET" ]] && { echo "Usage: $0 <process_name>"; exit 1; }

echo "=== Process Monitor for '$TARGET' ==="
printf "%-8s %-10s %-6s %-10s %-10s %s\n" \
    "PID" "STATE" "CPU%" "VM_SIZE" "RSS" "COMMAND"

for pid_dir in /proc/[0-9]*; do
    pid=$(basename "$pid_dir")
    [[ -f "$pid_dir/stat" ]] || continue
    cmdline=""
    [[ -f "$pid_dir/cmdline" ]] &&
        cmdline=$(tr '\0' ' ' < "$pid_dir/cmdline" 2>/dev/null)
    [[ "$cmdline" == *"$TARGET"* ]] || continue

    read -r p comm state ppid pgrp session tty_nr tpgid flags \
        minflt cminflt majflt cmajflt utime stime cutime cstime \
        priority nice num_threads itrealvalue starttime vsize rss \
        < "$pid_dir/stat" 2>/dev/null

    comm=${comm#\({}; comm=${comm%\}}
    page_size=$(getconf PAGESIZE)
    rss_kb=$((rss * page_size / 1024))
    vsize_mb=$((vsize / 1024 / 1024))

    printf "%-8s %-10s %-6s %-10s %-10s %s\n" \
        "$pid" "$state" "N/A" "${vsize_mb}MB" "${rss_kb}KB" "$cmdline"
done
```

Exercise 3: RAII File Descriptor Wrapper (C++ OOPs)

```

// FileDescriptor.hpp – RAII wrapper for POSIX file descriptors
#ifndef FILE_DESCRIPTOR_HPP
#define FILE_DESCRIPTOR_HPP
#include <unistd.h>
#include <fcntl.h>
#include <stdexcept>
#include <string>

class FileDescriptor {
private:
    int fd_;
public:
    explicit FileDescriptor(const std::string& path,
                           int flags = O_RDONLY) {
        fd_ = open(path.c_str(), flags);
        if (fd_ == -1)
            throw std::runtime_error("Failed to open: " + path);
    }
    ~FileDescriptor() { if (fd_ != -1) close(fd_); }
    // Copy: DELETED (FDs cannot be safely copied)
    FileDescriptor(const FileDescriptor&) = delete;
    FileDescriptor& operator=(const FileDescriptor&) = delete;
    // Move: transfer ownership
    FileDescriptor(FileDescriptor&& other) noexcept
        : fd_(other.fd_) { other.fd_ = -1; }
    FileDescriptor& operator=(FileDescriptor&& other) noexcept {
        if (this != &other) {
            if (fd_ != -1) close(fd_);
            fd_ = other.fd_; other.fd_ = -1;
        }
        return *this;
    }
    int get() const { return fd_; }
};
#endif

```

Exercise 4: Log Parser with Regex (System Diagnostics)

```
#!/usr/bin/env python3
"""syslog_analyzer.py – Production log analysis tool"""
import re
from collections import Counter, defaultdict

SYSLOG_PATTERN = re.compile(
    r'^(?P<timestamp>\w{3}\s+\d{1,2}\s+\d{2}:\d{2}:\d{2})\s+'
    r'(?P<hostname>\S+)\s+'
    r'(?P<process>\w+)(?:\[(?P<pid>\d+)\])?:\s+'
    r'(?P<message>.*)$'
)

def parse_syslog(filepath):
    entries = []
    with open(filepath, 'r', encoding='utf-8', errors='replace') as f:
        for line_num, line in enumerate(f, 1):
            line = line.strip()
            if not line: continue
            match = SYSLOG_PATTERN.match(line)
            if not match: continue
            entries.append({
                'timestamp': match.group('timestamp'),
                'hostname': match.group('hostname'),
                'process': match.group('process'),
                'pid': match.group('pid'),
                'message': match.group('message'),
                'line_num': line_num
            })
    return entries

def analyze_errors(entries):
    error_entries = [e for e in entries
                     if 'error' in e['message'].lower()]
    hourly_errors = defaultdict(int)
    process_errors = Counter()
    for entry in error_entries:
        try:
            hour = entry['timestamp'].split()[2].split(':')[0]
            hourly_errors[hour] += 1
        except IndexError: pass
    process_errors[entry['process']] += 1
    return {
        'total_errors': len(error_entries),
        'hourly_distribution': dict(sorted(hourly_errors.items())),
        'top_processes': process_errors.most_common(3)
    }
```

Actionable Learning Resources

Topic	Resource	Exact Location
Socket Programming & sendfile()	YouTube: The Coding Train — TCP/IP basics	Search "The Coding Train socket programming python"
Zero-Copy & sendfile() Deep Dive	YouTube: ByteByteGo — "Zero Copy"	Search "ByteByteGo zero copy"
Linux File Descriptors	YouTube: Neso Academy — "File Descriptors in Linux"	youtube.com/@nesoacademy
Process Management	YouTube: Neso Academy — "Process States", "Fork System Call"	Neso Academy OS playlist
Git Internals	Documentation: Git Book Chapter 10	git-scm.com/book/en/v2/Git-Internals-Plumbing-and-Porcelain
C++ Vtable	YouTube: The Cherno — "Virtual Functions in C++"	youtube.com/watch?v=oIV2KchSyGQ
C++ RAII & Rule of Five	YouTube: The Cherno — "Constructors and Destructors" series	The Cherno C++ playlist
Linux procs	Documentation: man 5 proc	man 5 proc on any Linux system
System Diagnostics	YouTube: Brendan Gregg — "Linux Performance Analysis in 60s"	Search "Brendan Gregg Linux performance"
Linux Networking	YouTube: Hussein Nasser — "TCP/IP Explained"	youtube.com/@hnasr

Interview Scenarios & Mock Diagnostics

Scenario 1: UnMTP Performance Bottleneck Under Pressure

Interviewer: "Your UnMTP tool uses `socket.sendfile()` for large files. Suppose a trader reports that during market hours, file transfers above 5GB are intermittently stalling for 30-60 seconds. The same transfer works fine after hours. Walk me through your diagnostic process."

Step 1 — Clarify the Problem (30 seconds):

"I want to confirm: Is the stalling on the sender side, receiver side, or both? Are we transferring over Wi-Fi or wired LAN? Is the 5GB threshold still optimal, or are we seeing issues right at the boundary? Are other network applications experiencing slowdowns during market hours?"

Step 2 — Hypothesize Root Causes (1 minute):

"Given the time-of-day correlation, I suspect network congestion or resource contention. My hypotheses: (1) **Network congestion**: More users on the same Wi-Fi/AP during market hours, reducing bandwidth. (2) **TCP buffer bloat**: 8MB socket buffers might be too large for a congested network. (3) **CPU throttling**: Device under load slows kernel TCP stack processing. (4) **sendfile() blocking**: sendfile() is blocking; if the TCP window closes due to congestion, it blocks the entire thread."

Step 3 — Diagnostic Commands (2 minutes):

```
# Check network throughput in real-time
ss -tin | grep -E '(bytes_acked|bytes_received|cwnd|rtt)'

# Monitor TCP congestion window and retransmits
netstat -s | grep -i retrans

# Check if sendfile is blocked on IO
strace -p <pid> -e trace=sendfile

# Check bandwidth available
iperf3 -c <receiver_ip>

# Check for packet loss
tcpdump -i any -c 100 | grep -i retrans

# System resource check
top -p <pid>; vmstat 1
```

Step 4 — Likely Fix (1 minute):

"(1) **Non-blocking I/O** with select()/poll(). (2) **Dynamic buffer sizing**: Reduce SO_SNDBUF/SO_RCVBUF based on measured RTT. (3) **Adaptive threshold**: Profile actual throughput and lower the sendfile() threshold if network conditions are poor. (4) **Progress reporting**: Add chunked sendfile() with offset tracking so users see progress instead of a frozen UI."

Step 5 — Prevention (30 seconds):

"I'd add telemetry — log transfer start time, duration, bytes transferred, and any EAGAIN/EWOULDBLOCK errors. This builds a dataset to predict optimal transfer parameters based on time of day and network conditions."

Scenario 2: Runux Command Injection Security Audit

Interviewer: "Your Runux tool translates natural language to shell commands. A security review flagged that LLM-generated commands could contain malicious payloads. Walk me through your current safety mechanisms and what you'd add for production deployment at a trading firm."

Step 1 — Acknowledge the Risk (30 seconds):

"This is a critical concern. LLMs are non-deterministic and can be prompt-injected. In a trading environment, a malicious command could exfiltrate data, modify positions, or disrupt systems. Defense in depth is essential."

Step 2 — Current Safeties in Runux (1 minute):

1. **Blast radius analysis** (blast_radius.py): Before execution, map what files/directories could be affected.
2. **Dry-run mode**: Users can preview the command before it runs.
3. **Sandboxing** (sandbox.py): Commands can optionally run in Docker containers.
4. **Snapshot/rollback** (snapshot.py): Filesystem state is captured before destructive operations.
5. **Plugin hooks**: run_before_translate and run_command_generated hooks allow injecting safety checks.

Step 3 — Production Hardening (2 minutes):

1. **Command whitelist**: Only allow commands from an approved set — no curl | bash, no eval, no rm -rf /.
2. **Argument sanitization**: Use shlex.quote() on all arguments; never use shell=True in subprocess.
3. **Network isolation**: Run in a sandbox with no outbound network access.
4. **Filesystem restrictions**: Use chroot or mount namespaces to limit filesystem visibility.
5. **Audit logging**: Every command, its source prompt, its blast radius, and its result are logged immutably.
6. **Approval gates**: Destructive operations require explicit human confirmation.
7. **Rate limiting**: Prevent automated exploitation via request throttling.

Step 4 — Technical Implementation (1 minute):

```
class SafetyEngine:
    CHECKS = [BlacklistCheck, WhitelistCheck, FilesystemScopeCheck,
              NetworkIsolationCheck, AuditLogger]
    def validate(self, command: str, context: dict) -> SafetyResult:
        for check in self.CHECKS:
            result = check.run(command, context)
            if not result.passed:
                return result # Fail fast
        return SafetyResult(passed=True)
```

Scenario 3: The-Stickerist OOM Prevention Pattern

Interviewer: "Your Stickerist bot processes images iteratively to prevent OOM. Explain the memory lifecycle of processing a 50-image Instagram carousel on a system with only 2GB RAM. How do you guarantee no OOM kill?"

Step 1 — Establish the Constraint (30 seconds):

"With 2GB RAM and a headless Chromium instance (200-400MB), we have limited headroom. Loading 50 images simultaneously would trigger the OOM killer. The key principle is: **process one image at a time, keep nothing in memory that isn't actively needed.**"

Step 2 — Walk Through the Lifecycle (2 minutes):

1. **Download:** Stream the image from Instagram directly to disk — never hold the full binary in a variable. Use `fs.createWriteStream()` in Node.js.
2. **Conversion:** Open with Sharp, convert to WebP, resize to 512x512, write output to a temp file.
3. **Quality optimization:** Check file size. If over 100KB, reduce quality and retry. Each iteration overwrites the previous temp file.
4. **Send via WhatsApp:** Stream the final file to the WhatsApp Web library.
5. **Immediate cleanup:** `fs.unlinkSync()` the temp file immediately after sending. Memory is freed before processing the next image.

Step 3 — Edge Cases (1 minute):

- **Zombie processes:** On startup, `killZombieBrowsers()` terminates stale Chromium processes from crashed sessions.
- **Global handlers:** `process.on('uncaughtException')` and `unhandledRejection` catch async errors.
- **Extremely large source images:** Sharp's `resize()` prevents loading massive images at full resolution.

Edge-Case Interview Questions (Week 1 Topics)

Topic	Question	Expected Answer
sendfile()	Can sendfile() work with UDP sockets?	No — sendfile() requires SOCK_STREAM (TCP). UDP is connectionless and message-oriented.
sendfile()	What happens if you modify the file while sendfile() is running?	Undefined behavior. The kernel reads from the page cache; modifications may or may not be reflected. Files should be immutable during transfer.
Tar streaming	Why use w mode instead of w for tar?	w is streaming mode — writes sequentially without seeking. Sockets are not seekable, so w would fail.
Process states	What's the difference between D (uninterruptible sleep) and S (interruptible sleep)?	D-state processes are waiting for I/O and cannot be killed (even with SIGKILL). S-state processes can be woken by signals.
File descriptors	After fork(), do parent and child share file descriptors?	They share the same underlying file description (offset, status flags). Both FDs point to the same entry in the kernel's open file table.

Git	What is a detached HEAD and when would you use it?	HEAD points directly to a commit instead of a branch. Used for inspecting old commits, temporary work, or creating tags.
C++	Why does a class with virtual functions have a larger sizeof?	The vptr (vtable pointer) is added as a hidden member — 8 bytes on 64-bit systems.
C++	What happens if a destructor throws an exception?	If a destructor throws during stack unwinding, <code>std::terminate()</code> is called immediately. Destructors should never throw.
Linux	Why is <code>wait()</code> necessary after <code>fork()</code> ?	Without <code>wait()</code> , the child becomes a zombie after exiting. The exit status occupies a slot in the process table until the parent reaps it.
Linux	What's the difference between <code>kill -9</code> and <code>kill -15</code> ?	<code>-9 (SIGKILL)</code> cannot be caught or ignored — forces immediate termination. <code>-15 (SIGTERM)</code> is a polite request to terminate; the process can clean up.

Week 2: Automation Scripting, Data Parsing & Core Logic

Sprint Goal: Master Python automation patterns, advanced shell scripting, regex for log parsing, and OOPs design. Build practical tools that mirror production operations tasks at a quantitative firm.

Day-by-Day Micro-Schedule

Day 8 — Python3: Generators, Iterators & Memory-Efficient Patterns (6 hours)

Time Block	Activity
2 hours	Theory: <code>__iter__</code> / <code>__next__</code> , generator functions (yield), generator expressions, yield from
2 hours	Theory: Memory profiling (tracemalloc, <code>sys.getsizeof()</code>), lazy evaluation, streaming data pipelines
2 hours	Coding: Re-implement The-Stickerist's image processing loop as a Python generator pipeline; profile memory usage

Day 9 — Python3: Context Managers, Decorators & Error Handling (6 hours)

Time Block	Activity
2 hours	Theory: <code>__enter__</code> / <code>__exit__</code> , <code>@contextmanager</code> , nested contexts, exception handling in context managers
2 hours	Theory: Decorator pattern (<code>@</code> syntax), <code>functools.wraps</code> , parameterized decorators, decorator stacking
2 hours	Coding: Write a <code>@retry</code> decorator with exponential backoff; write a <code>@timed</code> decorator for performance profiling

Day 10 — Regular Expressions: Deep Mastery for Log Parsing (6 hours)

Time Block	Activity
2 hours	Theory: Character classes, quantifiers, anchors, groups (capturing/non-capturing), lookahead/lookbehind
2 hours	Theory: <code>re</code> module functions (<code>match</code> , <code>search</code> , <code>findall</code> , <code>finditer</code> , <code>sub</code> , <code>split</code> , <code>compile</code>), flags (<code>re.IGNORECASE</code> , <code>re.MULTILINE</code> , <code>re.DOTALL</code>)
2 hours	Coding: Write regex patterns to parse: Apache access logs, JSON log lines, stack traces, and Python tracebacks

Day 11 — Shell Scripting: Advanced Bash for Production (6 hours)

Time Block	Activity
2 hours	Theory: Arrays, associative arrays, string manipulation (<code>\${var//pattern/replacement}</code>), parameter expansion
2 hours	Theory: Process substitution <code><()</code> , command substitution <code>\$()</code> , arithmetic <code>(())</code> , set -euo pipefail
2 hours	Coding: Write a deployment script that validates environment, checks disk space, backs up current version, deploys new version, health-checks, and rolls back on failure

Day 12 — OOPs Design: Python Classes, Inheritance & Patterns (6 hours)

Time Block	Activity
2 hours	Theory: <code>@dataclass</code> , <code>__slots__</code> , <code>__repr__</code> , <code>__eq__</code> , <code>__hash__</code> , <code>__post_init__</code> , frozen dataclasses
2 hours	Theory: Mixins, abstract base classes (<code>abc.ABC</code> , <code>@abstractmethod</code>), factory pattern, singleton pattern
2 hours	Coding: Design a LogParser class hierarchy: <code>base LogParser</code> → <code>SyslogParser</code> , <code>JsonLogParser</code> , <code>ApacheLogParser</code> using ABCs

Day 13 — String & Array Manipulation: Interview Core (6 hours)

Time Block	Activity
2 hours	Theory: Python string methods (immutable), <code>str.join()</code> , slicing, <code>collections.Counter</code> , <code>defaultdict</code>
2 hours	Theory: Two-pointer technique, sliding window, prefix/suffix arrays, string building patterns
2 hours	Coding: Solve LeetCode string/array problems (see drill below)

Day 14 — Week 2 Consolidation: Build a Production Log Analyzer (5 hours)

Time Block	Activity
3 hours	Build the Production Log Analyzer mini-project (full spec in Core Concepts)
2 hours	Review all Week 2 concepts; write 3 new regex patterns for your own projects' log formats

Core Concepts to Master

1. Python Generators & Memory-Efficient Patterns

How Generators Work Internally:

- A generator function (with `yield`) returns a **generator iterator** when called.
- When `__next__()` is called, the function runs until it hits `yield`, then **freezes its stack frame** and returns the value.
- On the next `__next__()`, execution resumes from after the `yield` — local variables are preserved.
- The generator object maintains its own frame on the heap, not the call stack.

Generator vs. List Comprehension Memory:

```
# List: creates all 1,000,000 integers in memory (~28MB)
squares_list = [x**2 for x in range(1_000_000)]

# Generator: creates an iterator (~112 bytes), yields one at a time
squares_gen = (x**2 for x in range(1_000_000))

# This is EXACTLY the pattern The-Stickerist uses:
# Instead of downloading all 50 images, process them one at a time
```

Streaming Pipeline Pattern:

```
def read_lines(filepath):          # Producer
    with open(filepath) as f:
        for line in f:
            yield line.strip()

def filter_errors(lines):          # Filter
    for line in lines:
        if 'ERROR' in line:
            yield line

def extract_timestamp(lines):      # Transformer
    for line in lines:
        match = re.search(r'\d{4}-\d{2}-\d{2}', line)
        if match:
            yield match.group()

# Pipeline: lazy, memory-constant regardless of file size
for ts in extract_timestamp(filter_errors(read_lines('app.log'))):
    print(ts)
```

2. Context Managers — The with Statement

Execution Flow:

1. `acquire_resource()` is called, returns a **context manager** object.
2. The CM's `__enter__()` is called; its return value is bound to the variable.
3. The block executes.
4. `__exit__(exc_type, exc_val, exc_tb)` is called — always, even on exceptions.
5. If `__exit__` returns `True`, the exception is suppressed; if `False` (or `None`), it propagates.

```
from contextlib import contextmanager
import time

@contextmanager
def timed_operation(name):
    start = time.perf_counter()
    try:
        yield # This is where the with block runs
    finally:
        elapsed = time.perf_counter() - start
        print(f"{name}: {elapsed:.3f}s")

# Usage:
with timed_operation("database_query"):
    result = db.execute("SELECT * FROM trades")
```

3. Decorators — Function Transformation

```
@my_decorator
def my_function(): pass

# Is syntactic sugar for:
def my_function(): pass
my_function = my_decorator(my_function)
```

Parameterized Decorator Pattern:


```

from functools import wraps
import time, random

def retry(max_attempts=3, backoff=2):
    def decorator(func):
        @wraps(func) # Preserves function metadata
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    if attempt == max_attempts:
                        raise
                    sleep_time = backoff ** attempt
                    time.sleep(sleep_time)
            return wrapper
        return decorator

@retry(max_attempts=5, backoff=1.5)
def fetch_market_data(): pass

```

Critical: Always Use @wraps — Without @wraps, the decorated function loses its `__name__`, `__doc__`, and `__module__`. This breaks introspection, logging, and debugging tools.

4. Regular Expressions — The Complete Mechanism

Pattern	Meaning	Example
.	Any character except newline	a.c matches "abc", "a_c"
\d	Digit [0-9]	\d{4} matches "2024"
\w	Word char [a-zA-Z0-9_]	\w+ matches "hello_world"
\s	Whitespace	\s+ matches spaces, tabs, newlines
^	Start of string/line	^ERROR matches "ERROR" at line start
\$	End of string/line	\.log\$ matches files ending in .log
*	0 or more (greedy)	a* matches "", "a", "aaa"
+	1 or more (greedy)	a+ matches "a", "aaa"
?	0 or 1	colou?r matches "color", "colour"
{n,m}	Between n and m	\d{2,4} matches "12", "123", "1234"
()	Capturing group	(\d+)-(\d+) captures two numbers
(?:)	Non-capturing group	(?:\d+) groups without capturing

(?=)	Positive lookahead	q(?=u) matches "q" only if followed by "u"
(?!)	Negative lookahead	q(?!u) matches "q" only if NOT followed by "u"

Regex Engine Internals:

- Python's re module uses a **backtracking NFA engine**.
- Greedy quantifiers (*, +) match as much as possible, then backtrack if needed.
- Lazy quantifiers (*?, +?) match as little as possible.
- **Catastrophic backtracking:** (a+)+ on "aaaaaaaaaaaaaaaaaaaaa!" can hang — avoid nested quantifiers on the same pattern.
- For complex parsing, re may not be sufficient — consider pyparsing or a proper parser.

Named Groups for Readable Parsing:

```
SYSLOG_RE = re.compile(r'''
    ^(?P<timestamp>\w{3}\s+\d{1,2}\s+\d{2}:\d{2}:\d{2})\s+
    (?P<hostname>\S+)\s+
    (?P<process>\w+)(?:(?P<pid>\d+)\s)?\s+
    (?P<message>.*)$
''', re.VERBOSE)
```

5. Advanced Bash — Production Scripting Patterns**set -euo pipefail — The Safety Trio:**

```
#!/bin/bash
set -euo pipefail
# -e: Exit immediately if any command exits non-zero
# -u: Treat unset variables as errors (no silent typos)
# -o pipefail: Pipeline fails if ANY command fails (not just the last)
```

Parameter Expansion:

```
name="/path/to/file.txt"
${name##*/}      # Remove longest match from start → "file.txt" (basename)
${name%/*}       # Remove shortest match from end → "/path/to" (dirname)
${name%.txt}     # Remove suffix → "/path/to/file"
${name:-default} # If unset/null, use "default"
${name:=default} # If unset/null, set AND use "default"
${name:?error}    # If unset/null, print "error" and exit
${#name}         # Length of string
${name^^}        # Uppercase
${name,,}        # Lowercase
```

Arrays and Associative Arrays:

```
# Indexed array
deploy_hosts=("web1" "web2" "web3")
echo "Count: ${#deploy_hosts[@]}"

# Associative array (hash map)
declare -A host_status
host_status["web1"]="healthy"
for host in "${!host_status[@]}"; do
    echo "$host: ${host_status[$host]}"
done
```

Trap — Signal Handling in Bash:

```
cleanup() {
    echo "Cleaning up temporary files..."
    rm -rf "$TMP_DIR"
    exit 0
}
trap cleanup EXIT INT TERM
```

6. OOPs Design Patterns — Python-Specific**Abstract Base Classes:**

```
from abc import ABC, abstractmethod

class LogParser(ABC):
    @abstractmethod
    def parse(self, line: str) -> dict: pass

    @abstractmethod
    def can_parse(self, line: str) -> bool: pass

class SyslogParser(LogParser):
    def can_parse(self, line: str) -> bool:
        return bool(re.match(r'^\w{3}\s+\d', line))
    def parse(self, line: str) -> dict:
        # Implementation
        pass
```

__slots__ — Memory Optimization:

- By default, Python instances store attributes in a `__dict__` (a hash map).
- `__slots__ = ('timestamp', 'level', 'message')` preallocates fixed slots.
- Benefits: ~40% memory reduction, faster attribute access, prevents adding new attributes.

Practical Tasks & Drills

Exercise 5: Production-Grade Retry Decorator with Exponential Backoff

```
#!/usr/bin/env python3
"""retry_decorator.py – Production retry pattern with exponential backoff"""
import time, random, functools
from typing import Type, Tuple, Optional

def retry(
    exceptions: Tuple[Type[Exception], ...] = (Exception,),
    max_attempts: int = 3,
    base_delay: float = 1.0,
    max_delay: float = 60.0,
    exponential_base: float = 2.0,
    jitter: bool = True,
    on_retry: Optional[callable] = None
):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            last_exception = None
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    last_exception = e
                    if attempt == max_attempts:
                        raise last_exception
                    delay = min(
                        base_delay * (exponential_base ** (attempt - 1)),
                        max_delay
                    )
                    if jitter:
                        delay *= (0.5 + random.random())
                    if on_retry:
                        on_retry(e, attempt, delay)
                    else:
                        print(f"Attempt {attempt} failed: {e}. "
                              f"Retrying in {delay:.2f}s...")
                        time.sleep(delay)
            raise last_exception
        return wrapper
    return decorator
```

Interview Talking Points: (1) Exponential backoff prevents overwhelming a struggling service (thundering herd). (2) Jitter randomizes retry times so multiple clients don't retry simultaneously. (3) The 'on_retry' callback allows logging/metrics without modifying the decorator. (4) We catch ONLY specified exceptions — unexpected errors fail fast (fail-safe). (5) `functools.wraps` preserves the original function's identity for introspection.

Exercise 6: Log Format Parser Using Named Regex Groups

Task: Write a Python module that can parse multiple log formats (syslog, Apache access log, JSON log) using regex with named groups. Include a factory method for auto-detection.

```
#!/usr/bin/env python3
"""multi_format_log_parser.py – Regex-based log parsing for production"""
import re, json
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Optional

@dataclass
class ParsedLogEntry:
    timestamp: Optional[str]
    level: str
    source: str
    message: str
    raw: str
    extras: dict

class LogParser(ABC):
    @abstractmethod
    def parse(self, line: str) -> Optional[ParsedLogEntry]: pass
    @abstractmethod
    def can_parse(self, line: str) -> bool: pass

class SyslogParser(LogParser):
    PATTERN = re.compile(
        r'^(?P<timestamp>\w{3}\s+\d{1,2}\s+\d{2}:\d{2}:\d{2})\s+'
        r'(?P<host>\S+)\s+'
        r'(?P<source>\w+)(?:\[(?P<pid>\d+)\])?\s+'
        r'(?P<message>.*?)$'
    )
    def can_parse(self, line: str) -> bool:
        return bool(self.PATTERN.match(line))
    def parse(self, line: str) -> Optional[ParsedLogEntry]:
        match = self.PATTERN.match(line)
        if not match: return None
        msg = match.group('message')
        level = 'INFO'
        if any(kw in msg.upper() for kw in ['ERROR', 'FATAL', 'CRITICAL']):
            level = 'ERROR'
        elif any(kw in msg.upper() for kw in ['WARN', 'WARNING']):
            level = 'WARN'
        return ParsedLogEntry(
            timestamp=match.group('timestamp'),
            level=level, source=match.group('source'),
            message=msg, raw=line,
            extras={'host': match.group('host'),
                    'pid': match.group('pid')})

class ApacheLogParser(LogParser):
    PATTERN = re.compile(
        r'^(?P<ip>[\d.]+\s+)(?P<ident>\S+)\s+(?P<auth>\S+)\s+'
        r'\[(?P<timestamp>[^\]]+)\]\s+'
    )
```

Exercise 7: Safe Deployment Script with Rollback

```
#!/bin/bash
# deploy.sh – Production deployment with rollback capability
set -euo pipefail

APP_NAME="trading-data-processor"
DEPLOY_DIR="/opt/${APP_NAME}"
BACKUP_DIR="/opt/backups/${APP_NAME}"
HEALTH_URL="http://localhost:8080/health"
MAX_HEALTH_WAIT=30
REQUIRED_DISK_MB=500

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'; NC='\033[0m'
log_info() { echo -e "${GREEN}[INFO]${NC} $*"; }
log_warn() { echo -e "${YELLOW}[WARN]${NC} $*"; }
log_error() { echo -e "${RED}[ERROR]${NC} $*" >&2; }

cleanup() {
    local exit_code=$?
    if [[ $exit_code -ne 0 ]]; then
        log_error "Deployment failed with exit code $exit_code"
        if [[ -f "${BACKUP_DIR}/.last_backup" ]]; then
            log_warn "Initiating rollback..."; rollback
        fi
    fi
}
trap cleanup EXIT

rollback() {
    local backup_timestamp=$(cat "${BACKUP_DIR}/.last_backup")
    local backup_path="${BACKUP_DIR}/${backup_timestamp}"
    [[ ! -d "$backup_path" ]] && { log_error "No backup found"; exit 1; }
    rm -rf "${DEPLOY_DIR}"; cp -a "$backup_path" "$DEPLOY_DIR"
    systemctl restart "$APP_NAME" || true
    log_info "Rollback complete"
}

preflight_checks() {
    log_info "Running preflight checks..."
    local available_mb=$(df -m "$DEPLOY_DIR" | awk 'NR==2 {print $4}')
    [[ "$available_mb" -lt "$REQUIRED_DISK_MB" ]] && {
        log_error "Insufficient disk space"; exit 1; }
    log_info "All preflight checks passed"
}

create_backup() {
    local timestamp=$(date +%Y%m%d_%H%M%S)
    local backup_path="${BACKUP_DIR}/${timestamp}"
    mkdir -p "$BACKUP_DIR"; cp -a "$DEPLOY_DIR" "$backup_path"
    echo "$timestamp" > "${BACKUP_DIR}/.last_backup"
    # Keep only last 5 backups
    ls -t "$BACKUP_DIR" | tail -n +6 | while read -r old; do

```


LeetCode Drill: String & Array Manipulation (Week 2)

#	Problem	Pattern	Why It Matters
1	Two Sum (Easy)	HashMap lookup	$O(n)$ vs $O(n^2)$; fundamental pattern for all pair-finding problems
2	Valid Anagram (Easy)	Counter/hash	Character frequency counting — used in log analysis
3	Group Anagrams (Medium)	HashMap of sorted strings	String manipulation + grouping — common in data categorization
4	Valid Parentheses (Easy)	Stack	Stack fundamentals — used in parsing, expression evaluation
5	Longest Substring Without Repeating Characters (Medium)	Sliding window	Two-pointer technique — critical for stream processing
6	Contains Duplicate (Easy)	HashSet	$O(1)$ lookup pattern — fundamental
7	Product of Array Except Self (Medium)	Prefix/suffix products	Array transformation without division — space optimization
8	Top K Frequent Elements (Medium)	HashMap + heap/bucket sort	Frequency analysis — exactly what log processing requires
9	Find All Anagrams in a String (Medium)	Sliding window + Counter	Pattern matching in streams — production monitoring use case
10	Decode String (Medium)	Stack	Nested structure parsing — common in config files, logs

Mini-Project: Production Log Analyzer

Objective: Build a command-line tool that analyzes production log files and generates a summary report. This project is for **skill-building only**.

Requirements:

1. Accept a log file path as argument.
2. Auto-detect log format (syslog, Apache, JSON).
3. Generate statistics: total lines, error count, warnings count, top 5 error-producing sources.
4. Output hourly error distribution.
5. Generate a summary report (text format).
6. Must use generators for memory efficiency.
7. Must use at least one decorator (`@timed` or `@retry`).

8. Must use regex with named groups.

```

log_analyzer/
├── __main__.py          # Entry point, argument parsing
├── parsers.py           # LogParser ABC + concrete parsers
├── analyzer.py          # Statistics computation using generators
├── reporters.py         # TextReporter class
├── decorators.py        # @timed, @retry
└── utils.py             # Helper functions

```

Actionable Learning Resources

Topic	Resource	Exact Location
Python Generators	YouTube: Corey Schafer — "Generators"	Search "Corey Schafer Python generators"
Python Context Managers	YouTube: Corey Schafer — "Context Managers"	Corey Schafer Python playlist
Python Decorators	YouTube: Corey Schafer — "Decorators"	Corey Schafer Python playlist
Python Dataclasses	YouTube: Corey Schafer — "Dataclasses"	Corey Schafer Python playlist
Regular Expressions	YouTube: Corey Schafer — "Regular Expressions" playlist (5 videos)	Search "Corey Schafer regular expressions"
Regex Interactive Practice	Website: regex101.com	regex101.com
Advanced Bash	YouTube: ProgrammingKnowledge — "Linux Shell Scripting"	Search "bash scripting advanced tutorial"
Bash Parameter Expansion	Documentation: GNU Bash Manual, Section 3.5.3	gnu.org/software/bash/manual
LeetCode Patterns	YouTube: NeetCode — "Arrays & Hashing" playlist	youtube.com/@NeetCode
Sliding Window Pattern	YouTube: NeetCode — "Sliding Window"	NeetCode Roadmap
Stack Pattern	YouTube: NeetCode — "Stack"	NeetCode Roadmap

Interview Scenarios & Mock Diagnostics

Scenario 4: Python Generator Pipeline for Market Data Streaming

Interviewer: "We receive market data as a continuous stream of JSON lines. Each line contains a ticker, timestamp, bid, ask, and volume. We need to compute a running VWAP (Volume-Weighted Average Price) per ticker in real-time. Memory is limited — we can't store all ticks. How would you design this in Python?"

Step 1 — Identify the Pattern (30 seconds):

"This is a streaming aggregation problem. The constraints are: unbounded input, limited memory, and real-time requirements. The solution is a **generator pipeline** that processes one tick at a time, maintaining only running aggregates per ticker."

Step 2 — Design the Data Structure (1 minute):

```
from dataclasses import dataclass
from typing import Dict

@dataclass
class RunningVWAP:
    cum_pv: float = 0.0 # Cumulative price * volume
    cum_vol: int = 0
    def add_tick(self, price: float, volume: int):
        self.cum_pv += price * volume
        self.cum_vol += volume
    @property
    def vwap(self) -> float:
        return self.cum_pv / self.cum_vol if self.cum_vol > 0 else 0.0
```

Memory is $O(\text{tickers})$, not $O(\text{ticks})$. For 10,000 tickers, this is ~160KB.

Step 3 — Build the Pipeline (2 minutes):

```

import json
from typing import Iterator

def read_ticks(filepath: str) -> Iterator[dict]:
    with open(filepath, 'r') as f:
        for line in f:
            yield json.loads(line)

def filter_valid(ticks: Iterator[dict]) -> Iterator[dict]:
    for tick in ticks:
        if all(k in tick for k in ('ticker', 'price', 'volume')):
            if tick['volume'] > 0 and tick['price'] > 0:
                yield tick

def compute_vwap(ticks: Iterator[dict]) -> Iterator[tuple]:
    vmaps: Dict[str, RunningVWAP] = {}
    for tick in ticks:
        ticker = tick['ticker']
        if ticker not in vmaps:
            vmaps[ticker] = RunningVWAP()
        vmaps[ticker].add_tick(tick['price'], tick['volume'])
        yield (ticker, vmaps[ticker].vwap)

# Wire them together:
for ticker, vwap in compute_vwap(filter_valid(read_ticks('market.jsonl'))):
    print(f'{ticker}: {vwap:.4f}')

```

Scenario 5: Regex Performance in Log Processing

Interviewer: "You have a Python script that parses 10GB of application logs daily using regex. It's currently taking 4 hours. How do you optimize it?"

Step 1 — Profile First (30 seconds):

```
python -m cProfile -s cumulative script.py
```

Step 2 — Regex Optimizations (2 minutes):

1. **Compile once:** Use `re.compile()` outside the loop, not `re.match()` inside.
2. **Use `re.search` vs `re.match` correctly:** match anchors to start; if you need to find anywhere, search is faster than `^.*pattern`.
3. **Avoid catastrophic backtracking:** Patterns like `(a+)+` or `(.)*` can hang.
4. **Use `finditer` not `findall`:** `findall` builds a list; `finditer` yields matches lazily.
5. **Use string methods when possible:** 'ERROR' in line is 10x faster than `re.search('ERROR', line)`.

Step 3 — Architecture Optimizations (1 minute):

1. **Parallel processing:** Use multiprocessing to parse chunks in parallel (CPU-bound).
2. **Line-by-line processing:** Use for line in file — never file.read() for large files.
3. **Filter early:** Check simple conditions (string contains) before running regex.
4. **Consider specialized tools:** grep/ripgrep for pre-filtering, awk for simple field extraction.

Edge-Case Interview Questions (Week 2 Topics)

Topic	Question	Answer
Generators	Can you iterate over a generator twice?	No. Generators are single-use iterators. After exhaustion, they're empty. Need to recreate or use <code>itertools.tee()</code> .
Generators	What does <code>yield from</code> do that for <code>x in gen: yield x</code> doesn't?	Same iteration, but <code>yield from</code> also transparently passes <code>.send()</code> and <code>.throw()</code> through to the sub-generator.
Context Managers	What happens if <code>__enter__</code> raises an exception?	The <code>with</code> block is never entered; <code>__exit__</code> is not called. The exception propagates normally.
Context Managers	Can you use multiple context managers in one <code>with</code> ?	Yes: with <code>cm1()</code> as <code>a</code> , <code>cm2()</code> as <code>b</code> : — both <code>__enter__</code> run, then both <code>__exit__</code> run in reverse order.
Decorators	What is <code>@functools.wraps</code> and why is it necessary?	Copies <code>__name__</code> , <code>__doc__</code> , <code>__module__</code> from the wrapped function to the wrapper. Without it, introspection and debugging break.
Regex	What's the difference between greedy <code>.*</code> and lazy <code>.+?</code> ?	Greedy matches as much as possible then backtracks; lazy matches as little as possible. <code>a.*b</code> on <code>"axxxbyyyb"</code> matches <code>"axxxbyyyb"</code> ; <code>a.+?b</code> matches <code>"axxxb"</code> .
Bash	What's the difference between <code>\$*</code> and <code>\$@</code> ?	Unquoted: same. Quoted: <code>"\$"</code> expands to one word (<code>"\$1 \$2 \$3"</code>); <code>"\$@"</code> expands to separate words (<code>"\$1" "\$2" "\$3"</code>). Always use <code>"\$@"</code> .
Python OOP	What's the difference between <code>__str__</code> and <code>__repr__</code> ?	<code>__repr__</code> is for developers (unambiguous, should be valid Python if possible); <code>__str__</code> is for users (readable). <code>print()</code> uses <code>__str__</code> ; REPL uses <code>__repr__</code> .
Python OOP	What is MRO (Method Resolution Order)?	The order in which Python looks for methods in a class hierarchy. Uses C3 linearization. Access via <code>ClassName.__mro__</code> .

Week 3: The Coding Baseline & Data Retrieval Pipelines

Sprint Goal: Establish solid DSA fundamentals through targeted LeetCode practice. Master intermediate SQL with JOINS, aggregations, window functions, and indexing. Build the mental muscle for coding under time pressure.

Day-by-Day Micro-Schedule

Day 15 — LeetCode: Arrays & HashMaps (The Foundation) (6 hours)

Time Block	Activity
1 hour	Theory: Hash table internals (separate chaining vs. open addressing), collision resolution, load factor, amortized $O(1)$
2 hours	Theory: Array vs. hash map trade-offs; when to use each; Python dict implementation (CPython 3.6+ insertion-ordered)
3 hours	Coding: Solve 4 LeetCode problems (Easy → Medium): Two Sum, Contains Duplicate, Group Anagrams, Top K Frequent Elements

Day 16 — LeetCode: String Manipulation & Two Pointers (6 hours)

Time Block	Activity
1 hour	Theory: String immutability in Python; string interning; str.join() vs. concatenation; slicing
2 hours	Theory: Two-pointer technique (slow/fast, left/right, sliding window); when and why it works
3 hours	Coding: Solve 4 problems: Valid Anagram, Longest Substring Without Repeating Characters, Valid Palindrome, Find All Anagrams in a String

Day 17 — LeetCode: Stack & Queue Patterns (6 hours)

Time Block	Activity
1 hour	Theory: Stack ADT (LIFO), queue ADT (FIFO); Python list as stack; collections.deque for $O(1)$ both-ends operations
2 hours	Theory: Monotonic stack pattern; using stacks for parsing, nearest greater/smaller element problems
3 hours	Coding: Solve 4 problems: Valid Parentheses, Min Stack, Daily Temperatures, Decode String

Day 18 — SQL: JOINS, Subqueries & Set Operations (6 hours)

Time Block	Activity
2 hours	Theory: INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN, CROSS JOIN; JOIN execution order; Cartesian products
2 hours	Theory: Correlated vs. non-correlated subqueries; EXISTS vs. IN; UNION vs. UNION ALL; INTERSECT; EXCEPT
2 hours	Coding: Write 8 SQL queries involving multi-table JOINS, nested subqueries, and set operations (see exercises below)

Day 19 — SQL: Aggregations, GROUP BY & Window Functions (6 hours)

Time Block	Activity
2 hours	Theory: COUNT, SUM, AVG, MIN, MAX; GROUP BY semantics; HAVING vs WHERE; NULL handling in aggregates
2 hours	Theory: Window functions (ROW_NUMBER, RANK, DENSE_RANK, LEAD, LAG, NTILE, PARTITION BY, ORDER BY, frame clauses)
2 hours	Coding: Write 8 SQL queries using aggregations and window functions (see exercises below)

Day 20 — SQL: Indexing, Query Optimization & Execution Plans (6 hours)

Time Block	Activity
2 hours	Theory: B-tree indexes, clustered vs. non-clustered, composite indexes, covering indexes, index selectivity
2 hours	Theory: Query execution plans (EXPLAIN, EXPLAIN ANALYZE); reading plans; common optimization patterns
2 hours	Coding: Analyze 5 query execution plans; optimize slow queries by adding appropriate indexes

Day 21 — Week 3 Consolidation: Timed Mock & SQL Drill (5 hours)

Time Block	Activity
2 hours	Timed LeetCode mock: 3 problems in 60 minutes (simulate interview pressure)
2 hours	Complete all SQL exercises from Days 18-20 without looking at solutions

1 hour

Review weak areas; re-solve any problems where you needed hints

Core Concepts to Master

1. Hash Tables — The Engine Behind Python dict

How Python Dictionaries Work (CPython 3.6+):

- Python dicts use a **combination of open addressing and separate chaining**.
- Internal structure: a sparse array of indices (the "hash table") + a dense array of entries (key-value pairs).
- The hash function: $\text{hash}(\text{key}) \rightarrow \text{integer} \rightarrow \text{index} = \text{hash} \& \text{mask}$ (where $\text{mask} = \text{table_size} - 1$).
- **Insertion-ordered since 3.6**: The dense array preserves insertion order; the sparse array only maps hash $\rightarrow$ position.
- **Resizing**: When load factor exceeds $2/3$, the table doubles in size and all entries are rehashed.
- Amortized time complexity: $O(1)$ for lookup, insert, delete.

Collision Resolution:

- When two keys hash to the same index, Python uses **open addressing with probing**.
- Probing sequence: $\text{index} = (5 * i + 1 + \text{perturb}) \% \text{size}$, where perturb is derived from the hash.
- This is more cache-friendly than separate chaining (which uses linked lists).

When Dicts Degrade:

- If all keys have the same hash (hash collision attack), lookups degrade to $O(n)$.
- Python 3.3+ mitigates this with **hash randomization**: PYTHONHASHSEED randomizes the hash function per process.

2. Two-Pointer Technique — Complete Pattern Map

Pattern 1: Slow & Fast Pointer (Floyd's Cycle Detection):

- Used for: cycle detection in linked lists, finding middle element, duplicate detection.
- Slow pointer moves 1 step, fast pointer moves 2 steps.
- If there's a cycle, fast will eventually meet slow. Mathematical proof: If cycle length is C , fast gains 1 per iteration; after C steps, fast laps slow.

Pattern 2: Left & Right Pointer (Converging):

- Used for: two-sum in sorted array, container with most water, palindrome check.
- Left starts at beginning, right at end; converge based on condition.
- Guarantees $O(n)$ for problems that would otherwise be $O(n^2)$.

Pattern 3: Sliding Window (Fixed/Variable Size):

- Used for: substring problems, subarray sum, consecutive sequences.
- Expand window by moving right pointer; contract by moving left pointer.
- The key invariant: window always represents a valid candidate solution.

3. Stack — Beyond Just LIFO**Monotonic Stack:**

- A stack that maintains elements in monotonically increasing or decreasing order.
- Used for: nearest greater element, nearest smaller element, largest rectangle in histogram.
- Pattern: For each element, pop smaller/larger elements, then push current.

Python Stack Implementation:

```
# Using list (simple but resizing cost)
stack = []
stack.append(item)    # Push — O(1) amortized
item = stack.pop()    # Pop — O(1)
item = stack[-1]      # Peek — O(1)

# Using collections.deque (preferred for large stacks — O(1) guaranteed)
from collections import deque
stack = deque()
stack.append(item)    # Push — O(1)
item = stack.pop()    # Pop — O(1)
```

4. SQL JOINS — Visual & Mechanistic Understanding

JOIN Type	Result
INNER JOIN	Only matching rows from both tables
LEFT JOIN	All rows from left table, matching rows from right (NULL if no match)
RIGHT JOIN	All rows from right table, matching rows from left (NULL if no match)
FULL OUTER JOIN	All rows from both tables (NULL where no match)
CROSS JOIN	Cartesian product — every row from A with every row from B (M x N rows)
SELF JOIN	Table joined with itself (use aliases)

Execution Order (Important!):

SQL logical processing order is NOT top-to-bottom: FROM + JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. This means: aliases defined in SELECT cannot be used in WHERE, but CAN be used in ORDER BY.

JOIN Algorithms:

Algorithm	When Used	Complexity
Nested Loop Join	Small tables, or when one table is very small	$O(M \times N)$
Hash Join	No index, medium-to-large tables	$O(M + N)$ build + probe
Merge Join	Both tables sorted on join key (or indexes)	$O(M + N)$
Index Nested Loop	One table has index on join column	$O(M \times \log(N))$

5. SQL Window Functions — The Game Changer

Window vs. GROUP BY:

- GROUP BY collapses rows into summary rows — you lose individual row detail.
- Window functions compute a value across a set of rows **while retaining individual rows**.
- Think of it as: "for each row, compute something using a window of related rows."

Function	What It Does	Example Use
ROW_NUMBER()	Unique sequential number (1, 2, 3...)	"Top N per group"
RANK()	Rank with gaps (1, 1, 3, 3, 5...)	"Top 3 without ties consuming slots"
DENSE_RANK()	Rank without gaps (1, 1, 2, 2, 3...)	"Competition rankings"
LEAD(col, n)	Value from N rows ahead	"Day-over-day change"
LAG(col, n)	Value from N rows behind	"Previous period comparison"
NTILE(n)	Divide rows into N equal buckets	"Quartile/percentile analysis"
FIRST_VALUE(col)	First value in window	"Opening price of the day"
LAST_VALUE(col)	Last value in window	"Closing price"

Frame Clause (The Window Within the Window):

```
ROWS BETWEEN 2 PRECEDING AND 2 FOLLOWING -- 5-row window centered on current row
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW -- Running total from start
ROWS BETWEEN 3 PRECEDING AND CURRENT ROW -- 4-row trailing window
RANGE BETWEEN INTERVAL '7' DAY PRECEDING AND CURRENT ROW -- Time-based window
```

6. SQL Indexing — How B-Trees Work

B-Tree Index Structure:

- A balanced tree where every leaf is at the same depth.
- Each node contains keys in sorted order plus pointers to child nodes.
- Typical fan-out: hundreds of keys per node (determined by page size, usually 8KB).
- For a million rows, tree height is only ~ 3 ($\log_{100}(1,000,000)$).
- **All operations (lookup, insert, delete) are $O(\log n)$.**

Clustered vs. Non-Clustered:

Property	Clustered	Non-Clustered
Data order	Table rows are physically sorted by index key	Separate structure; index contains key + row pointer
Number allowed	One per table	Many per table
Lookup speed	Fastest (data is right there)	Requires second lookup (or bookmark lookup)
INSERT cost	High (may require reordering rows)	Low (just add to index)

Composite Index Column Order:

- Query WHERE a = 1 AND b = 2 can use index on (a, b) or (b, a).
- But WHERE a = 1 can only use index (a, b), NOT (b, a) — **leftmost prefix rule**.
- Put the most selective column first; put columns used in equality conditions before range conditions.

Covering Index:

- An index that contains ALL columns needed for a query.
- The database can satisfy the query entirely from the index — no table lookup needed.
- Example: If query is SELECT name, salary FROM employees WHERE dept = 'IT', then index on (dept, name, salary) is covering.

Practical Tasks & Drills

Exercise 8: HashMap Design (Understand the Internals)

```
#!/usr/bin/env python3
"""hashmap_impl.py – Simple hash map implementation to understand internals"""
from typing import Optional, Iterator

class HashMap:
    """Simple hash map using separate chaining for collision resolution."""
    class _Node:
        __slots__ = ('key', 'value', 'next')
        def __init__(self, key, value):
            self.key = key; self.value = value
            self.next: Optional[HashMap._Node] = None

    def __init__(self, initial_capacity: int = 16):
        self._capacity = initial_capacity
        self._size = 0
        self._table = [None] * self._capacity
        self._load_factor_threshold = 0.75

    def _hash(self, key) -> int:
        return hash(key) & (self._capacity - 1)

    def _resize(self):
        old_table = self._table
        self._capacity *= 2
        self._size = 0
        self._table = [None] * self._capacity
        for node in old_table:
            while node:
                self.put(node.key, node.value)
                node = node.next

    def put(self, key, value):
        if self._size / self._capacity >= self._load_factor_threshold:
            self._resize()
        idx = self._hash(key)
        node = self._table[idx]
        while node:
            if node.key == key:
                node.value = value; return
            node = node.next
        new_node = self._Node(key, value)
        new_node.next = self._table[idx]
        self._table[idx] = new_node
        self._size += 1

    def get(self, key):
        idx = self._hash(key)
        node = self._table[idx]
        while node:
            if node.key == key: return node.value
            node = node.next
```

Exercise 9: LeetCode Walkthrough — Top K Frequent Elements

Problem: Given an integer array `nums` and an integer `k`, return the `k` most frequent elements.

Bucket Sort Approach ($O(n)$):

```
def top_k_frequent(nums, k):
    """
    Bucket sort approach:  $O(n)$  time,  $O(n)$  space
    Key insight: The frequency of any element is at most  $n$ .
    Create  $n+1$  buckets where bucket[i] contains all elements with frequency  $i$ .
    """
    from collections import Counter, defaultdict

    freq_map = Counter(nums) # {num -> count}
    buckets = defaultdict(list)
    for num, freq in freq_map.items():
        buckets[freq].append(num)

    result = []
    for freq in range(len(nums), 0, -1):
        if freq in buckets:
            result.extend(buckets[freq])
            if len(result) >= k:
                return result[:k]
    return result[:k]
```

Approach	Time	Space	When to Use
Sorting	$O(n \log n)$	$O(n)$	General case, simple code
Min-Heap (size k)	$O(n \log k)$	$O(n)$	When $k \ll n$
Bucket Sort	$O(n)$	$O(n)$	When guaranteed $O(n)$ is needed

Exercise 10: LeetCode Walkthrough — Daily Temperatures (Monotonic Stack)

```
def daily_temperatures(temperatures):  
    """  
    Monotonic decreasing stack pattern.  
    Key insight: For each day, we want the next day with higher temperature.  
    We use a stack that stores indices with DECREASING temperatures.  
    When we see a temperature warmer than stack top, we've found the answer  
    for the stacked day(s) – pop and calculate day difference.  
    O(n) because each index is pushed and popped at most once.  
    """  
    n = len(temperatures)  
    answer = [0] * n  
    stack = [] # Stack of indices with decreasing temperatures  
  
    for i, temp in enumerate(temperatures):  
        while stack and temperatures[stack[-1]] < temp:  
            prev_day = stack.pop()  
            answer[prev_day] = i - prev_day  
        stack.append(i)  
    return answer
```

Exercise 11: SQL — Trading Database Schema & Queries

Schema Setup:

```
CREATE TABLE trades (  
    trade_id    INT PRIMARY KEY,  
    ticker      VARCHAR(10) NOT NULL,  
    trade_time  TIMESTAMP NOT NULL,  
    side        CHAR(1) CHECK (side IN ('B', 'S')),  
    quantity    INT NOT NULL CHECK (quantity > 0),  
    price       DECIMAL(18, 8) NOT NULL,  
    trader_id   INT NOT NULL,  
    desk        VARCHAR(20) NOT NULL  
);  
  
CREATE TABLE traders (  
    trader_id   INT PRIMARY KEY,  
    name        VARCHAR(100) NOT NULL,  
    desk        VARCHAR(20) NOT NULL,  
    start_date  DATE NOT NULL  
);  
  
CREATE TABLE eod_prices (  
    ticker      VARCHAR(10) NOT NULL,  
    price_date  DATE NOT NULL,  
    open_price  DECIMAL(18, 8),  
    high_price  DECIMAL(18, 8),  
    low_price   DECIMAL(18, 8),  
    close_price DECIMAL(18, 8),  
    volume      BIGINT,  
    PRIMARY KEY (ticker, price_date)  
);
```

Query Set 1: JOINS and Basic Aggregation:


```

-- Q1: Total volume traded per ticker, sorted by volume desc
SELECT ticker, SUM(quantity) AS total_volume
FROM trades GROUP BY ticker ORDER BY total_volume DESC;

-- Q2: All trades with trader names (INNER JOIN)
SELECT t.trade_id, t.ticker, t.price, t.quantity, tr.name AS trader_name
FROM trades t INNER JOIN traders tr ON t.trader_id = tr.trader_id;

-- Q3: All traders and their trades, including traders with no trades (LEFT JOIN)
SELECT tr.name, tr.desk, COUNT(t.trade_id) AS trade_count
FROM traders tr LEFT JOIN trades t ON tr.trader_id = t.trader_id
GROUP BY tr.trader_id, tr.name, tr.desk;

-- Q4: Tickers traded on '2024-06-15' that also traded on '2024-06-14'
SELECT DISTINCT t1.ticker FROM trades t1
WHERE t1.trade_time::DATE = '2024-06-15'
  AND EXISTS (
    SELECT 1 FROM trades t2
    WHERE t2.ticker = t1.ticker
    AND t2.trade_time::DATE = '2024-06-14');

-- Q5: Average trade size per desk, only for desks with > 100 trades
SELECT desk, AVG(quantity) AS avg_trade_size, COUNT(*) AS trade_count
FROM trades GROUP BY desk HAVING COUNT(*) > 100
ORDER BY avg_trade_size DESC;

```

Query Set 2: Window Functions:

```

-- Q6: Running total volume per ticker (ordered by time)
SELECT trade_id, ticker, trade_time, quantity,
       SUM(quantity) OVER (
         PARTITION BY ticker ORDER BY trade_time
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_volume
FROM trades;

-- Q7: Rank traders by total notional (price x qty) traded
SELECT trader_id,
       SUM(price * quantity) AS total_notional,
       RANK() OVER (ORDER BY SUM(price * quantity) DESC) AS notional_rank
FROM trades GROUP BY trader_id;

-- Q8: For each trade, show the previous trade's price for the same ticker (LAG)
SELECT trade_id, ticker, trade_time, price,
       LAG(price) OVER (PARTITION BY ticker ORDER BY trade_time) AS prev_price,
       price - LAG(price) OVER (PARTITION BY ticker ORDER BY trade_time) AS price_change
FROM trades;

-- Q9: Top 3 most active trading days per desk
WITH daily_stats AS (
  SELECT desk, trade_time::DATE AS trade_date,
         COUNT(*) AS trade_count,
         RANK() OVER (PARTITION BY desk ORDER BY COUNT(*) DESC) AS rnk
  FROM trades GROUP BY desk, trade_time::DATE
)
SELECT desk, trade_date, trade_count FROM daily_stats
WHERE rnk <= 3 ORDER BY desk, rnk;

```

Query Set 3: Indexing and Optimization:

```

CREATE INDEX idx_trades_ticker_time ON trades(ticker, trade_time);
CREATE INDEX idx_trades_trader ON trades(trader_id);
CREATE INDEX idx_trades_desk_time ON trades(desk, trade_time);

-- Composite covering index for the running volume query
CREATE INDEX idx_trades_ticker_time_qty ON trades(ticker, trade_time, quantity);

-- Check if query uses the index
EXPLAIN ANALYZE
SELECT ticker, SUM(quantity) AS total_volume
FROM trades
WHERE trade_time >= '2024-06-01' AND trade_time < '2024-07-01'
GROUP BY ticker;

```

Actionable Learning Resources

Topic	Resource	Exact Location
Hash Tables Deep Dive	YouTube: Abdul Bari — "Hashing"	Search "Abdul Bari hashing"
Two Pointer Technique	YouTube: NeetCode — "Two Pointers"	NeetCode Roadmap
Sliding Window	YouTube: NeetCode — "Sliding Window"	NeetCode Roadmap
Stack Patterns	YouTube: NeetCode — "Stack"	NeetCode Roadmap
LeetCode Patterns	Website: NeetCode Roadmap	neetcode.io/roadmap
SQL JOINS Visual	Website: SQL Joins Visualizer	sql-joins.leopard.in.ua
SQL Window Functions	YouTube: Decomplexify — "SQL Window Functions"	Search "Decomplexify SQL window functions"
SQL Indexing	YouTube: Hussein Nasser — "Database Indexing"	Hussein Nasser channel
B-Tree Explanation	YouTube: Michael Sambol — "B-Trees"	Search "Michael Sambol B-Tree"
SQL Practice	Website: SQLBolt (free interactive)	sqlbolt.com
Advanced SQL Practice	Website: HackerRank SQL	hackerrank.com/domains/sql

Interview Scenarios & Mock Diagnostics

Scenario 6: Real-Time Trade Duplicate Detection

Interviewer: "Our trading system receives trade executions from multiple venues. Occasionally, the same trade is reported twice with a slightly different timestamp (within 1 millisecond). We need to detect and remove duplicates in real-time. Design the data structure and algorithm."

Step 1 — Clarify Requirements (30 seconds):

"What's the volume — trades per second? Can we define a unique trade key (e.g., trade ID + venue)? What's the tolerance window? Do we need exact deduplication or probabilistic (Bloom filter)?"

Step 2 — Design (2 minutes):

```

from collections import deque
import time

class TradeDeduplicator:
    def __init__(self, window_ms=1000):
        self.window_ms = window_ms
        self.trades = deque() # (timestamp_ms, trade_signature)
        self.seen = set()      # Fast lookup

    def _clean_expired(self, now_ms):
        while self.trades and self.trades[0][0] < now_ms - self.window_ms:
            old_ts, old_sig = self.trades.popleft()
            self.seen.discard(old_sig)

    def is_duplicate(self, trade_id, venue, ticker, qty, price, timestamp_ms):
        signature = (trade_id, venue, ticker, qty, price)
        self._clean_expired(timestamp_ms)
        if signature in self.seen:
            return True
        self.seen.add(signature)
        self.trades.append((timestamp_ms, signature))
        return False

```

Step 3 — Handle Scale (1 minute):

"At 100,000 trades/second with a 1-second window, we're storing 100K signatures. Each signature is ~40 bytes → ~4MB memory. For higher scale, I'd shard by ticker symbol across multiple deduplicator instances."

Scenario 7: SQL Query Optimization for Trade Reporting

Interviewer: "We have a trades table with 500 million rows. A trader runs this query daily and it takes 45 minutes: `SELECT ticker, AVG(price) FROM trades WHERE trade_time >= '2024-01-01' GROUP BY ticker;` How do you make it run in under 10 seconds?"

Step 1 — Add the Right Index (30 seconds):

```

CREATE INDEX idx_trades_time_ticker_price
ON trades(trade_time, ticker, price);

```

This covering composite index allows an index-only scan.

Step 2 — Verify with EXPLAIN (30 seconds):

Run `EXPLAIN ANALYZE` to confirm the query planner uses an Index Only Scan instead of a Sequential Scan. If it doesn't, check if table statistics are up to date (`ANALYZE trades`).

Step 3 — Pre-aggregation (1 minute):

```
CREATE MATERIALIZED VIEW daily_ticker_stats AS
SELECT ticker, trade_time::DATE AS trade_date,
       AVG(price) AS avg_price,
       COUNT(*) AS trade_count,
       SUM(quantity) AS total_volume
FROM trades GROUP BY ticker, trade_time::DATE;

CREATE INDEX idx_daily_stats ON daily_ticker_stats(ticker, trade_date);

-- Refresh nightly
REFRESH MATERIALIZED VIEW daily_ticker_stats;
```

This turns a 500M-row aggregation into a sub-second query on a pre-computed summary.

Step 4 — Partitioning (30 seconds):

"If data is append-only (typical for trades), I'd also consider table partitioning by trade_time (monthly or daily). This enables partition pruning — the query only scans relevant partitions."

Edge-Case Interview Questions (Week 3 Topics)

Topic	Question	Answer
HashMap	What happens if two different objects have the same hash code?	Collision. Python resolves via probing; Java uses chaining. Lookup becomes $O(n)$ in worst case, but average remains $O(1)$.
HashMap	Can you use a mutable object as a dictionary key?	No — if the object mutates after insertion, its hash changes, and you can't retrieve it. Only immutable/hashable objects can be keys.
Two Pointers	When does the two-pointer technique NOT work?	When elements need to be compared with non-adjacent elements (e.g., finding a subset sum), or when the array isn't sortable.
Stack	What's the difference between a stack and a queue?	Stack is LIFO (Last In First Out); queue is FIFO (First In First Out). Stack: undo operations; Queue: BFS, task scheduling.
SQL	What's the difference between WHERE and HAVING?	WHERE filters rows before grouping; HAVING filters groups after aggregation. WHERE can't use aggregate functions; HAVING can.
SQL	Can you have multiple columns in a PARTITION BY clause?	Yes: PARTITION BY desk, ticker creates windows within each unique (desk, ticker) combination.
SQL	What's a covering index and why is it fast?	An index that contains all columns referenced in a query. The database doesn't need to look up the actual table — everything is in the index.

Index	When does an index hurt performance?	On very small tables (index overhead > benefit), on columns with low cardinality (most rows match), during heavy write workloads (index maintenance cost).
-------	--------------------------------------	--

Week 4: Full Production Simulation & Interview Polish

Sprint Goal: Synthesize all knowledge into interview-ready responses. Master troubleshooting methodology, complexity analysis, behavioral storytelling, and time-constrained problem-solving. Finish with a capstone project that ties everything together.

Day-by-Day Micro-Schedule

Day 22 — Troubleshooting Methodology & Production Scenarios (6 hours)

Time Block	Activity
2 hours	Theory: Structured debugging framework (reproduce → hypothesize → isolate → fix → verify → prevent); divide-and-conquer; binary search on configurations
2 hours	Theory: Linux performance troubleshooting: Brendan Gregg's USE method (Utilization, Saturation, Errors); latency analysis; flame graphs
2 hours	Practice: Walk through 5 production incident scenarios; verbalize your diagnostic process aloud

Day 23 — Time/Space Complexity Analysis & Big-O (6 hours)

Time Block	Activity
2 hours	Theory: Big-O, Big-Omega, Big-Theta definitions; amortized analysis; Master theorem for recurrences; space complexity vs. auxiliary space
2 hours	Theory: Common complexity classes with examples; best/average/worst case analysis; how to analyze recursive algorithms
2 hours	Practice: Analyze time and space complexity of all your project code; analyze 10 LeetCode solutions

Day 24 — Mock Technical Interview Round 1 (6 hours)

Time Block	Activity
3 hours	Mock Interview Set A: 1 coding problem (45 min), 1 systems troubleshooting scenario (45 min), 1 project deep-dive (30 min)
3 hours	Self-review: Record yourself solving problems; identify filler words, unclear explanations, time management issues

Day 25 — Mock Technical Interview Round 2 (6 hours)

Time Block	Activity
3 hours	Mock Interview Set B: 1 SQL optimization problem (30 min), 1 Linux scripting exercise (45 min), 1 system design discussion (45 min)
3 hours	Self-review: Compare Round 1 vs. Round 2 performance; document improvements

Day 26 — Behavioral Interview Mastery (5 hours)

Time Block	Activity
2 hours	Theory: STAR method (Situation, Task, Action, Result); D.E. Shaw cultural values; crafting compelling narratives
2 hours	Write and rehearse 8 behavioral stories (see scenarios below)
1 hour	Practice "Tell me about yourself" pitch (2-minute version)

Day 27 — System Design Mini-Sessions (6 hours)

Time Block	Activity
2 hours	Theory: System design framework (requirements → capacity → API → data model → high-level design → detailed design → trade-offs)
2 hours	Practice: Design a log aggregation system; design a monitoring/alerting system; design a configuration management service
2 hours	Practice: Explain trade-offs for each design; defend your choices under pressure

Day 28 — Final Review & Weak Area Blitz (6 hours)

Time Block	Activity
3 hours	Re-solve the 5 hardest problems from Weeks 1-3 without hints
3 hours	Re-read all interview scenarios; practice verbalizing answers to a mirror or recorder

Day 29 — Capstone Mini-Project Build (8 hours)

Time Block	Activity
4 hours	Build the ProductionOps Dashboard capstone project (full blueprint at the end of this document)
4 hours	Test, polish, and document the project

Day 30 — Final Simulation & Rest (4 hours)

Time Block	Activity
2 hours	Final full mock interview (all question types back-to-back)
2 hours	Light review of flashcards; early rest — mental freshness > cramming

Core Concepts to Master

1. Structured Troubleshooting — The FOTA Methodology

The Framework (Memorize This):

1. REPRODUCE → Can you make the problem happen consistently?
2. HYPOTHESIZE → Form 2-3 educated guesses about root cause
3. ISOLATE → Design experiments that distinguish between hypotheses
4. FIX → Apply the minimum viable fix
5. VERIFY → Confirm the fix works and doesn't break anything else
6. PREVENT → Add monitoring/tests/process to prevent recurrence

Brendan Gregg's USE Method:

For every resource (CPU, Memory, Disk, Network), check: **Utilization** (how busy is it? 0-100%), **Saturation** (how much work is queued/waiting?), **Errors** (any error counts?).

Example — High CPU Diagnosis:

```
# 1. Check utilization
mpstat -P ALL 1 # Per-CPU utilization

# 2. Check saturation (run queue)
vmstat 1 # 'r' column = runnable processes waiting for CPU

# 3. Find which processes
pidstat -u 1 # Per-process CPU usage

# 4. Drill into the hot process
perf top -p <pid> # Hot functions
strace -cp <pid> # Syscall frequency

# 5. If it's Python: profile the code
python -m cProfile -s cumulative script.py
```

2. Big-O Analysis — Complete Reference

Class	Name	Example
$O(1)$	Constant	Array lookup, hash map access, stack push/pop
$O(\log n)$	Logarithmic	Binary search, B-tree lookup
$O(n)$	Linear	Array scan, linked list traversal
$O(n \log n)$	Linearithmic	Merge sort, quicksort (average), heap sort
$O(n^2)$	Quadratic	Bubble sort, nested loops
$O(2^n)$	Exponential	Subset enumeration, brute-force TSP
$O(n!)$	Factorial	Permutation enumeration

Amortized Analysis:

- Python list `append()`: $O(1)$ amortized — occasionally $O(n)$ when resizing, but rare enough that n appends cost $O(n)$ total.
- Dict insertion: $O(1)$ amortized — occasionally $O(n)$ when rehashing.

Master Theorem (for recurrences):

For $T(n) = a \cdot T(n/b) + f(n)$ where $a \geq 1$, $b > 1$:

- If $f(n) = O(n^{(\log_b(a) - \epsilon)})$: $T(n) = \Theta(n^{\log_b(a)})$
- If $f(n) = \Theta(n^{\log_b(a)})$: $T(n) = \Theta(n^{\log_b(a)} \cdot \log n)$
- If $f(n) = \Omega(n^{(\log_b(a) + \epsilon)})$ and $af(n/b) \leq cf(n)$: $T(n) = \Theta(f(n))$

Common cases: Merge sort: $T(n) = 2T(n/2) + O(n) \rightarrow \text{Case 2} \rightarrow \Theta(n \log n)$. Binary search: $T(n) = T(n/2) + O(1) \rightarrow \text{Case 2} \rightarrow \Theta(\log n)$.

3. System Design — The FOTA Perspective

Framework for Answering:

1. **REQUIREMENTS** — Functional: What does it do? Non-functional: Scale, latency, availability, durability.
2. **CAPACITY ESTIMATION** — QPS, storage, bandwidth. Back-of-envelope math.
3. **API DESIGN** — Endpoints, request/response formats.
4. **DATA MODEL** — Schema, relationships.
5. **HIGH-LEVEL DESIGN** — Components and their interactions. Draw a diagram.
6. **DETAILED DESIGN** — Algorithm/data structure choices. Trade-off analysis.
7. **OPERATIONAL CONCERNS** — Monitoring, alerting, deployment. Failure modes and recovery.

Key D.E. Shaw System Design Topics:

- **Real-time data pipelines:** Market data ingestion, processing, distribution.
- **Monitoring and alerting:** Metrics collection, threshold-based alerting, on-call rotation.
- **Configuration management:** Feature flags, A/B testing infrastructure.
- **Log aggregation:** Centralized logging, search, alerting on log patterns.
- **Service discovery:** How services find each other in a dynamic environment.

Trade-off Language (Use These Phrases):

- "The trade-off here is latency vs. consistency..."
- "I'd optimize for read throughput at the cost of write complexity..."
- "This adds operational complexity but improves availability..."
- "The simpler approach is easier to debug during incidents..."
- "We can start with X and evolve to Y when scale demands it..."

4. Behavioral Interview — The STAR Method

Structure Every Story:

- **SITUATION** (20%): Brief context — what was the project/team/constraint?
- **TASK** (10%): What was YOUR specific responsibility?
- **ACTION** (50%): What did YOU do? Use "I" not "we". Be specific.
- **RESULT** (20%): Quantified outcome. What metric improved? What did you learn?

D.E. Shaw Values to Demonstrate:

1. **Intellectual curiosity:** "I was curious about X, so I dug deeper and found Y."
2. **Comfort with ambiguity:** "The requirements weren't clear, so I defined the problem by..."
3. **Quiet confidence:** Calm, structured thinking under pressure.
4. **Collaboration:** Working with difficult stakeholders, cross-team projects.
5. **Resilience:** Projects that failed and what you learned.

Practical Tasks & Exercises

Exercise 12: Complexity Analysis of Your Own Projects

UnMTP Analysis:

```
def send_file(sock, filepath):
    filesize = os.path.getsize(filepath)
    sock.sendall(struct.pack('!Q', filesize)) # O(1)
    with open(filepath, 'rb') as f:
        if filesize >= BIG_FILE_THRESHOLD:
            sock.sendfile(f) # O(1) syscall; transfers in kernel
        else:
            with tarfile.open(fileobj=sock, mode='w|') as tar:
                tar.add(filepath) # O(n) where n = file size
```

Time: $O(n)$ where n = bytes transferred. Each byte is touched exactly once (or zero times for `sendfile()`). **Space:** $O(1)$ auxiliary — we don't buffer the file in memory. The 8MB socket buffer is constant. For a 10GB file, $O(1)$ space means it runs on a Raspberry Pi.

Runux Analysis:

```
def analyze_blast_radius(command):
    affected_paths = []
    for root, dirs, files in os.walk(start_path): # O(n) filesystem entries
        for f in files:
            affected_paths.append(os.path.join(root, f))
    return affected_paths
```

Time: $O(n)$ where n = filesystem entries. **Space:** $O(n)$ for storing all paths — could be optimized to $O(1)$ if we only return a count or $O(k)$ if we return only top- k . Optimization: Use a generator to yield paths instead of building a list.

Exercise 13: Production Incident — Verbal Walkthrough Practice

Scenario: "It's 9:30 AM. Market open. A trader reports that the trade reporting dashboard is not loading — it's timing out after 30 seconds. The dashboard queries a PostgreSQL database that stores today's trades. Yesterday it was fine. Walk me through your diagnosis."

Your Verbal Response (memorize the structure):

"Okay, the key clues are: it started today, it was fine yesterday, and it's happening at market open — peak load time. My hypotheses are:

First, connection pool exhaustion. At market open, many traders load the dashboard simultaneously. If the connection pool is sized for average load, it could be saturated. I'd check: `SELECT count(*) FROM pg_stat_activity;` — if it's near `max_connections`, that's the issue. Fix: increase pool size or add connection pooling (PgBouncer).

Second, a missing or corrupted index. If an index on `trade_time` was dropped or corrupted overnight, the query would scan the entire table. I'd run `EXPLAIN ANALYZE` on the dashboard query and look for 'Seq Scan' on a large table. Fix: `REINDEX INDEX idx_trades_time;` or recreate the index.

Third, table bloat or lock contention. If a long-running transaction from yesterday is holding locks, new queries queue up. I'd check: `SELECT * FROM pg_locks WHERE NOT granted;` and kill the offending long-running query.

My first action would be to check `pg_stat_activity` — it tells me connection count, active queries, and locks within 10 seconds. Based on that, I'd know which hypothesis is correct and act immediately."

Exercise 14: Behavioral Story Bank

Prepare these 8 stories using STAR. Write them out in full, then practice telling them in 90 seconds each.

Story 1: Tell me about yourself.

"I'm a systems-focused engineer with deep experience in automation and low-level optimization. My strongest project is UnMTP, where I implemented zero-copy file transfer using kernel bypass syscalls. I also built Runux, a natural-language-to-shell translator with safety features like blast-radius analysis and sandboxed execution. What drives me is the challenge of making systems that are both powerful and safe to operate in production."

Story 2: Describe a challenging bug you solved.

- **S:** The-Stickerist was intermittently failing on large carousels (30+ images).
- **T:** Find the root cause without being able to reproduce it consistently.
- **A:** Added memory profiling (`process.memoryUsage()` logging) and discovered OOM kills at ~1.8GB. Switched to explicit `fs.unlinkSync()` after each send and added `global.gc()` hints between carousels.

- **R:** Zero failures since. Memory stays under 400MB regardless of carousel size.

Story 3: Tell me about a time you had to learn something quickly.

- **S:** Needed to implement `socket.sendfile()` for UnMTP but had never used kernel bypass before.
- **T:** Understand zero-copy semantics and benchmark against traditional I/O.
- **A:** Read the Linux `sendfile(2)` man page, traced the Python `socket.sendfile()` C implementation, and built a benchmark script using `perf stat`.
- **R:** Achieved 40% CPU reduction on large transfers. The benchmark data justified the 5GB threshold heuristic.

Story 4: Describe a time you disagreed with a design decision.

- **S:** In Runux, a teammate wanted to execute LLM-generated commands directly without confirmation.
- **T:** Advocate for safety mechanisms without blocking the feature.
- **A:** Built a prototype showing 3 injection scenarios the LLM produced during testing. Proposed blast-radius analysis + dry-run mode + explicit confirmation for destructive ops.
- **R:** Safety features adopted. The tool has been used 500+ times with zero incidents.

Story 5: Tell me about a project that failed.

- **S:** Initially built UnMTP using Python's `http.server` for file serving.
- **T:** Handle 10GB+ files efficiently.
- **A:** HTTP approach buffered entire files in memory, causing OOM at ~2GB. Pivoted to raw sockets with `sendfile()` and streaming tar, rewriting ~60% of the code.
- **R:** Final version handles 50GB+ files on devices with 2GB RAM. Taught me to validate architecture against worst-case scale early.

Story 6: How do you handle multiple competing priorities?

- **S:** During exam week, three active projects plus coursework.
- **T:** Make progress on all without sacrificing quality.
- **A:** Used time-blocking (2-hour deep-work sprints), prioritized by deadline and dependency, and automated repetitive tasks.
- **R:** All three projects completed. UnMTP got 3 GitHub stars and a feature request from a real user.

Story 7: Why D.E. Shaw?

"Three reasons: First, the Systems group's work — building tools for state-of-the-art data centers, managing infrastructure at the scale of a top quant firm — is exactly the intersection of low-level systems and high-stakes operations I want to work on. Second, D.E. Shaw's interview guide explicitly values intellectual curiosity and comfort with ambiguity — those are my strengths. Third, the FOTA role sits at the intersection of technology and front-office trading, which means the work directly impacts the business."

Story 8: What's your greatest technical weakness?

"My formal DSA background is at a beginner level — I'm self-taught through projects rather than competitive programming. I've been addressing this systematically: I've solved 30+ LeetCode problems focused on arrays, hash maps, and strings, and I can now analyze time/space complexity confidently. What I bring in exchange is deep systems intuition from building production tools that handle real hardware constraints."

Actionable Learning Resources

Topic	Resource	Exact Location
Linux Performance	YouTube: Brendan Gregg — "Linux Performance Tools" full playlist	Search "Brendan Gregg Linux performance"
USE Method	Blog: Brendan Gregg's USE Method	brendangregg.com/usemethod.html
Big-O Analysis	YouTube: Abdul Bari — "Time Complexity"	Search "Abdul Bari time complexity"
Master Theorem	YouTube: Abdul Bari — "Master Theorem"	Abdul Bari algorithms playlist
System Design	YouTube: ByteByteGo — System Design playlist	youtube.com/@ByteByteGo
Behavioral Interviews	YouTube: Jeff H Sipe — "Behavioral Interview"	Search "Jeff H Sipe behavioral interview STAR"
D.E. Shaw Culture	PDF: D.E. Shaw Interviewing Guide	deshawindia.com/assets/articles/D_E_Shaw_Interviewing_Guide.pdf
Mock Interviews	Website: Pramp (free mock interviews)	pramp.com

Interview Scenarios & Mock Diagnostics

Scenario 8: Linux Memory Pressure Under Trading Load

Interviewer: "A trading application running on Linux starts experiencing slowdowns at market open. free -h shows plenty of free memory, but top shows high %wa (I/O wait). vmstat shows increasing si/so (swap in/swap out). What's happening and how do you fix it?"

Diagnosis:

This is a classic **memory pressure leading to swap thrashing** scenario. The key insight is that `free` shows free memory, but the issue isn't total memory — it's that active memory is being swapped out due to memory pressure from other processes.

1. `vmstat 1` shows `si/so` (swap in/out) increasing — confirms swapping.
2. `sar -B 1` shows high `pgpgin/pgpgout` — pages being swapped actively.
3. `ps aux --sort=-%mem | head` identifies the top memory consumers.
4. `/proc/<pid>/status` → `VmRSS` (resident), `VmSwap` (swapped) for the trading app.

Fix (in order of speed):

1. **Immediate:** `swapoff -a && swapon -a` — forces swap back to RAM (temporary, risky).
2. **Better:** Identify and stop/renice the memory-heavy non-critical process.
3. **Proper:** Configure `vm.swappiness` lower (default 60 → 10); add more RAM; use cgroups to limit the batch process's memory; pin the trading app's memory with `mlock()`.

Scenario 9: Explain Your UnMTP Project in 3 Minutes

Your Prepared Response:

"UnMTP is a bi-directional file transfer tool I built to move files between Android and Windows without USB cables or cloud services. It handles two key scenarios differently based on file size.

For large files — 5GB and above — I use Python's `socket.sendfile()`, which wraps the Linux `sendfile(2)` syscall. This bypasses userspace entirely: data moves directly from kernel page cache to the network socket. The result is zero CPU overhead for the actual transfer — the process just makes one syscall and the kernel handles everything.

For smaller files, I use a streaming tar pipe. I create a tar archive in `w|` mode — that's the streaming variant that doesn't seek — and pipe it directly through the socket. This bundles metadata and multiple small files efficiently without intermediate disk writes.

Key design decisions: The 5GB threshold is a heuristic I validated through benchmarking. I tuned socket buffers to 8MB for sustained throughput on gigabit Wi-Fi. Cross-platform path normalization handles Windows backslash-to-forward-slash translation automatically.

The hardest part was handling connection failures gracefully. I implemented an interactive retry loop with IP re-entry, which is important because mobile hotspots can be flaky. I also validate write permissions by creating and deleting a test file before initiating transfer — this catches permission issues early instead of failing mid-transfer."

Scenario 10: System Design — Log Aggregation for Trading Systems

Interviewer: "Design a system that collects logs from 1000 trading servers, makes them searchable within 30 seconds, and alerts on error patterns."

Requirements: 1000 servers x 10MB/hour each = 10GB/hour = ~3MB/second ingestion. 30-second search latency for recent logs. Alert on error rate spikes, specific error patterns. Retain 30 days.

High-Level Design:

```
Trading Servers → Log Agents → Kafka → Log Processors → Elasticsearch → Kibana
                                     ↓
                               Alert Manager
```

Components:

1. **Log Agent** (Fluentd/Filebeat) on each server: tails log files, batches, compresses, sends to Kafka.
2. **Kafka**: Durability buffer; handles ingestion spikes; partitions by service name.
3. **Log Processors**: Parse raw logs (regex/json extraction), enrich with metadata, index into Elasticsearch.
4. **Elasticsearch**: Inverted index for full-text search; hot nodes for recent data, warm nodes for older.
5. **Alert Manager**: Runs scheduled queries on Elasticsearch; triggers PagerDuty/Slack on threshold breaches.

Trade-offs:

- "Elasticsearch gives us sub-second search but adds operational complexity. For a smaller scale, Loki is simpler but less powerful."
- "Kafka adds latency (batching) but provides durability and backpressure handling — critical for trading where we can't lose audit logs."

Edge-Case Interview Questions (Week 4 Topics)

Topic	Question	Answer
Troubleshooting	You SSH to a server and it's completely unresponsive. top won't run. What do you do?	Use <code>echo t > /proc/sysrq-trigger</code> (SysRq magic) to get emergency console output; check <code>dmesg</code> from serial console; if completely hung, check hardware (IPMI/iLO logs).
Troubleshooting	What's the difference between load average and CPU utilization?	CPU utilization = % of time CPU is busy. Load average = average number of runnable OR uninterruptible processes over 1/5/15 min. High load + low CPU = I/O or lock contention.
Big-O	What's the amortized complexity of Python list append?	$O(1)$ amortized. When capacity is exceeded, Python allocates a new array ~1.125x the size and copies. Over n appends, total copies = $O(n)$, so average per append is $O(1)$.
Big-O	Is binary search always $O(\log n)$?	Only on random-access structures (arrays). On linked lists, binary search is $O(n)$ because you can't jump to the middle in $O(1)$.

System Design	How do you choose between SQL and NoSQL?	SQL: ACID transactions, complex joins, structured schema, strong consistency. NoSQL: horizontal scaling, flexible schema, high write throughput, eventual consistency acceptable.
System Design	How do you handle a cascading failure?	Circuit breakers (fail fast), bulkheads (isolate failures), rate limiting, graceful degradation (serve stale data), emergency switches (feature flags to disable features).
Behavioral	Why should we hire you over someone from a more prestigious university?	Focus on demonstrated production experience: "I've built and shipped three systems tools that handle real hardware constraints. My projects show I can learn complex topics and deliver working code."

Capstone Mini-Project Blueprint: ProductionOps Dashboard

Project Name: *prodops-dashboard* | **Objective:** *A command-line dashboard that monitors local system health, analyzes log files in real-time, and generates alerts — combining ALL skills from this plan. This project is designed to demonstrate production operations skills in your interview.*

Architecture

```
prodops-dashboard/  
├── __main__.py           # CLI entry point with argparse  
├── system_monitor.py     # CPU, memory, disk, network metrics (Week 1)  
├── log_analyzer.py       # Multi-format log parsing with regex (Week 2)  
├── alert_engine.py       # Threshold-based alerting with decorators (Week 2)  
├── database.py           # SQLite storage for metrics history (Week 3)  
├── dashboard.py          # Terminal UI with curses/rich (Week 4)  
├── config.py             # Configuration management  
└── utils.py              # Retry decorator, timing decorator, etc.
```

Feature Specifications

1. System Metrics Collection (system_monitor.py)

```
class SystemMonitor:  
    def get_cpu_percent(self) -> float:  
        """Read from /proc/stat. Calculate user+nice+system time."""  
  
    def get_memory_info(self) -> dict:  
        """Read from /proc/meminfo. Return total, available, used, percent."""  
  
    def get_disk_usage(self, path='/') -> dict:  
        """Use os.statvfs. Return total, used, free, percent."""  
  
    def get_network_io(self) -> dict:  
        """Read from /proc/net/dev. Return bytes_sent, bytes_recv."""  
  
    def get_load_average(self) -> tuple:  
        """Read from /proc/loadavg. Return 1min, 5min, 15min."""
```

2. Log Analysis (log_analyzer.py)

- Reuse the LogParserFactory from Week 2 Exercise 6.
- Add a tail -f mode that watches a log file and reports errors in real-time.

- Use generators for memory efficiency.

3. Alert Engine (alert_engine.py)

```
from dataclasses import dataclass
from typing import Callable
import time

@dataclass
class AlertRule:
    name: str
    metric: str          # e.g., "cpu_percent"
    threshold: float      # e.g., 90.0
    comparator: str       # ">", "<", "=="
    cooldown_seconds: int = 300 # Don't alert more than once per 5 min

class AlertEngine:
    def __init__(self):
        self.rules: list[AlertRule] = []
        self.last_alerted: dict[str, float] = {} # rule_name -> timestamp

    def add_rule(self, rule: AlertRule):
        self.rules.append(rule)

    def check(self, metrics: dict) -> list[str]:
        """Check all rules against current metrics. Return triggered alerts."""
        triggered = []
        now = time.time()
        for rule in self.rules:
            if rule.metric not in metrics:
                continue
            value = metrics[rule.metric]
            comparator_fn = {
                '>': lambda a, b: a > b,
                '<': lambda a, b: a < b,
                '>=': lambda a, b: a >= b,
                '<=': lambda a, b: a <= b,
            }[rule.comparator]
            if comparator_fn(value, rule.threshold):
                last = self.last_alerted.get(rule.name, 0)
                if now - last >= rule.cooldown_seconds:
                    triggered.append(
                        f"ALERT: {rule.name} - {rule.metric}={value:.1f} "
                        f"({rule.comparator} {rule.threshold})")
                    self.last_alerted[rule.name] = now
        return triggered
```

4. Database Storage (database.py)

- SQLite database for storing metrics history.
- Table: metrics (timestamp REAL, metric_name TEXT, value REAL).

- Query: average CPU over last hour, peak memory in last 24 hours.
- Use context managers for connection handling.

5. Terminal Dashboard (dashboard.py)

- Use the rich library (Python) for beautiful terminal output.
- Display: CPU gauge, memory bar, disk usage, recent alerts, top 5 error sources from logs.
- Auto-refresh every 2 seconds.

6. Decorators (utils.py)

```

from functools import wraps
import time

def retry(max_attempts=3, delay=1.0):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except Exception:
                    if attempt == max_attempts - 1:
                        raise
                    time.sleep(delay)
            return wrapper
        return decorator

def timed(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"[TIMED] {func.__name__}: {elapsed:.4f}s")
        return result
    return wrapper

def rate_limited(max_calls, period_seconds):
    def decorator(func):
        calls = []
        @wraps(func)
        def wrapper(*args, **kwargs):
            now = time.time()
            calls[:] = [c for c in calls if now - c < period_seconds]
            if len(calls) >= max_calls:
                return None
            calls.append(now)
            return func(*args, **kwargs)
        return wrapper
    return decorator

```

Sample Usage

```
# Start monitoring with dashboard
python -m prodops-dashboard --mode dashboard --log-file /var/log/syslog

# Run log analysis only
python -m prodops-dashboard --mode analyze --log-file app.log --format auto

# Add custom alert rules
python -m prodops-dashboard --alert "cpu_percent > 90" --alert "disk_usage > 85"
```

Skills Demonstrated

Week	Skills Applied
Week 1	/proc filesystem reading, system diagnostics, process monitoring
Week 2	Generator pipelines, regex parsing, decorators (retry, timed, rate_limit), context managers
Week 3	SQL aggregations, window functions (for metrics history), OOP design (ABC, factory)
Week 4	Troubleshooting methodology, system design thinking, production-ready error handling

Interview Narrative

"I built prodops-dashboard as a capstone project to synthesize everything I learned. It reads system metrics directly from /proc, parses logs with auto-detected formats using regex named groups, stores history in SQLite for trend analysis, and uses a decorator-based alert engine with rate limiting and cooldowns. The whole thing runs in a terminal with live updates. It's the kind of tool I'd want on my desk as a production engineer — lightweight, zero dependencies beyond Python standard library + rich, and immediately useful for diagnosing issues."

Appendix A: D.E. Shaw Behavioral Question Bank

Prepare answers for these using the STAR method. Write out full responses, then practice delivering each in 90 seconds.

#	Question	Primary Focus Area
1	Tell me about yourself.	Personal narrative, project highlights, motivation
2	Why D.E. Shaw? Why the FOTA role specifically?	Company research, role alignment, career goals
3	Describe a challenging technical problem you solved.	Technical depth, problem-solving process
4	Tell me about a time you had to learn something quickly.	Adaptability, learning agility
5	Describe a time you disagreed with a teammate. How did you resolve it?	Collaboration, conflict resolution
6	Tell me about a project that failed. What did you learn?	Resilience, self-awareness, growth mindset
7	How do you prioritize when you have multiple deadlines?	Time management, prioritization framework
8	Describe a time you went above and beyond.	Ownership, initiative, impact
9	What's your greatest technical strength? Weakness?	Self-awareness, honest assessment
10	Where do you see yourself in 3-5 years?	Ambition, alignment with role trajectory
11	Tell me about a time you had to debug something under time pressure.	Pressure handling, systematic debugging
12	How do you stay current with technology?	Intellectual curiosity, continuous learning
13	Describe a time you had to explain a technical concept to a non-technical person.	Communication skills, empathy
14	What would you do if you were given an ambiguous problem with no clear solution?	Comfort with ambiguity, structured thinking
15	Tell me about a time you made a decision with incomplete information.	Judgment, risk assessment, decisiveness

Appendix B: Linux Command Quick Reference

Process Management

```
ps aux | grep <name>           # Find processes by name
pgrep -f <pattern>             # PID of matching processes
pkill -f <pattern>             # Kill matching processes
nice -n 10 <cmd>               # Run with lower priority
renice +5 -p <pid>             # Change priority of running process
```

Memory

```
top -o %MEM                    # Sort by memory
smem -r                        # Memory per user (PSS)
cat /proc/<pid>/status | grep -E 'VmRSS|VmSwap'
```

Disk

```
iostat -xz 1                   # Extended I/O stats, ignore zeroes
df -ih                          # Inode usage (often the real limit)
du -sh * | sort -rh | head     # Top disk consumers
fuser -v <file>                 # What process has file open
```

Network

```
ss -tulpn                      # Listening sockets with PIDs
tcpdump -i any -c 100          # Capture 100 packets
lsof -i :8080                  # What's using port 8080
nc -zv <host> <port>           # Test connectivity
```

Filesystem

```
find . -name '*.log' -mtime +7 -delete # Delete old logs
find . -type f -size +100M             # Files over 100MB
rsync -avz --delete src/ dst/          # Sync directories
```

Performance

```
perf top -p <pid>              # Hot functions in a process
strace -cp <pid>                # Syscall summary
vmstat 1 10                     # 10 samples, 1-second interval
sar -u 1 10                     # CPU utilization samples
```

Appendix C: Complexity Cheat Sheet — Your Projects

Project	Component	Time	Space	Interview Talking Point
UnMTP	sendfile() transfer	O(n) bytes	O(1)	Zero-copy kernel bypass
UnMTP	Tar streaming	O(n) bytes	O(1)	Streaming without buffering
UnMTP	Path resolution	O(d) path depth	O(1)	os.path.normpath
Runux	Command execution	O(c) command	O(1)	subprocess isolation
Runux	Blast radius analysis	O(f) files	O(f)	Filesystem walk
Runux	Plugin hook execution	O(p) plugins	O(1)	Observer pattern
Stickerist	Image processing	O(i) per image	O(i)	Iterative OOM prevention
Stickerist	Quality adjustment	O(k) iterations	O(i)	Bounded loop (quality > 10)
Stickerist	Browser cleanup	O(z) zombies	O(1)	Startup process hygiene
Log Analyzer	Regex parsing	O(n) per line	O(1)	Compiled regex, generator pipeline
HashMap	Insert/lookup/delete	O(1) amortized	O(n)	Separate chaining
Trade Dedup	Check duplicate	O(1)	O(w) window	Sliding window + hash set

Final Advice: This plan is aggressive by design. If you have only 20 days, compress by doing Days 1-2 together and skipping the capstone project (Days 29-30). If you have 30 days, build the capstone — it's your strongest interview asset. Every morning, spend 15 minutes reviewing the previous day's material before starting new content. Every evening, spend 15 minutes writing down 3 things you learned and 1 thing that confused you. Good luck — you've got this.

Battle Plan generated for: D.E. Shaw FOTA Role | Candidate: Aryan Kushwaha
Plan Version: 1.0 | Optimized for: 20-30 day preparation window